

TAPI v2.1.5 Reference Implementation Agreement

TR-548

TAPI Streaming

Version 1.2
December 2025

Document Type: Technical Recommendation

Disclaimer

THIS SPECIFICATION IS PROVIDED "AS IS" WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NONINFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, OR ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

Any marks and brands contained herein are the property of their respective owners.

Linux Foundation

548 Market St, PMB 57274, San Francisco, California, 94104-5401 US

www.linuxfoundation.org

Copyright © 2025 The Linux Foundation®. All rights reserved.

The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our [Policies](#) page. [Privacy Policy](#) | [Trademark Usage](#).

The Linux Foundation is a non-profit organization that supports the development of Linux and other open source projects. The Linux Foundation provides a neutral, trusted hub for developers to code, manage, and scale open technology projects.

Table of Contents

Disclaimer	2
Document History	6
1 Introduction	7
1.1 General introduction.....	7
1.2 Introduction to this document.....	7
1.3 Specification.....	7
2 Overview.....	8
2.1 Essential features and benefits.....	8
2.2 TAPI application	9
3 Summary of key considerations	10
3.1 Overview	10
3.2 Existing RESTCONF notification mechanism (described in [LF TR-547])	10
3.3 TAPI Streaming available in 2.1.3	10
3.4 Stream content.....	10
3.5 TAPI Application in detail	11
3.6 Summary of Streaming Characteristics	12
3.7 Supported and available streams	13
3.7.1 Supported stream type	13
3.7.2 Available Streams	16
3.8 Streaming approach and log strategy	16
3.8.1 Log storage strategy	17
3.8.1.1 Compacted log.....	17
3.8.1.2 Truncated log.....	17
3.8.1.3 Full history log.....	18
3.8.1.4 Full history with periodic baseline log	18
3.8.2 Log record strategy	18
3.8.2.1 Whole entity on change	18
3.8.2.2 Change only.....	19
3.8.2.3 Whole entity periodic	19
3.9 Using the stream	19
3.9.1 Streaming the context.....	19
3.9.1.1 Effect of streaming approach and compacted log characteristics on alignment	20
3.9.1.2 Preparing to connect.....	20
3.9.1.3 Initial connection	20
3.9.1.4 Tombstone (Delete) retention passed	20
3.9.1.5 Compaction delay passed	20
3.9.1.6 (Eventual) Consistency achieved	21
3.9.1.7 Degraded performance.....	21
3.9.1.8 Need for realignment	21
3.9.1.9 Summary	21

3.9.2	Future combination considerations (by example).....	21
3.9.2.1	Many clients.....	21
3.9.2.2	Many views (and many clients, a few per view)	22
3.9.2.3	Many short-lived clients	22
3.9.2.4	Live measurements	22
3.9.2.5	Threshold Crossing.....	22
3.9.2.6	Periodic measurement data.....	22
3.9.2.7	Bulk Performance Monitoring (PM) data	23
3.10	Record content.....	23
3.10.1	Considering parent-address	27
3.11	Considering order/sequence and cause/effect	27
3.11.1	Time	27
3.11.2	Backend stream details	28
3.12	The Context	28
3.13	Handling changes in the Context.....	28
3.14	Reporting change.....	28
3.15	Model implications	29
3.16	System engineering	29
3.17	Eventual Consistency and Fidelity	29
3.17.1	Eventual Consistency	29
3.17.2	Fidelity	29
3.17.3	Related stream characteristics	30
3.18	Stream Monitor	30
3.19	Solution structure – Architecture Options	31
3.19.1	Full compacted log.....	31
3.19.2	Emulated compaction	32
3.19.3	Comparing the full compacted log and the emulated compacted log	34
4	Using the compacted log approach for alarm reporting	34
4.1	Specific alarm characteristics - raising/clearing an alarm.....	34
4.2	Key Features of an alarm solution (example usage)	34
4.3	Log strategy	34
4.4	Alarm behavior	35
4.5	Condition detector and alarm structure.....	36
4.6	Alarm Identifier and location	39
4.7	Alarm tombstone behavior	39
4.8	Time	40
4.9	Detected Condition normalization	40
4.10	Meaningful detection (device or any other source).....	40
5	Use Cases	41
5.1	Use Case General Considerations	41
5.1.1	TAPI Context	41
5.1.2	Underlying behavior	41
5.1.3	Model conformance	41
5.1.4	Use Case Overview	43
5.2	Streaming infrastructure use cases	43
5.2.1	Use Case ST-0.1: Get Auth Token.....	43

5.2.2	Use Case ST-0.2: Discover supported and available streams, then select available streams	43
5.2.3	Use Case ST-0.3: Connect to Stream and align - new client	44
5.2.4	Use Case ST-0.4: Client maintains idle connection	46
5.2.5	Use Case ST-0.5: Provider delivers event storm (or slow client) – bad day	46
5.2.6	Use Case ST-0.6: Provider delivers extreme event storm (or very slow client) – very bad day	47
5.2.7	Use Case ST-0.7: Short loss of communications	48
5.2.8	Use Case ST-0.8: Long loss of communications requiring realignment	49
5.2.9	Use Case ST-0.9: Client requires realignment	51
5.3	Building and operating a stream on a provider	51
5.3.1	Use Case ST-1.1: Provider initializes and operates a stream	52
5.3.2	Use Case ST-1.2: Provider recovers a stream after internal loss	53
5.3.3	Use Case ST-1.3: Provider recovers a stream after an upgrade	54
5.4	Client maintains alignment – Example strategies and approaches	55
5.4.1	Use Case ST-2.1: Client aligns with a stream	55
5.4.2	Use Case ST-2.2: Client realigns	56
5.4.3	Use Case ST-2.3: Client performs a stream audit	58
5.5	Gaining and maintaining Alignment with individual network resources	58
5.5.1	Use Case ST-3.1: Client maintains alignment with all instances of a class (e.g., Node) in a context	58
5.5.2	Use Case ST-3.2: Client maintains alignment with all alarms in the context	59
5.6	Dealing with the whole context of resources	60
5.6.1	Use Case ST-4.1: Client maintains alignment with all resources in the context	60
5.6.2	Use Case ST-4.2: In a resilient solution the Controller a client is connected to becomes unavailable	60
5.7	Connectivity Service Lifecycle	60
5.7.1	Use Case ST-5.1: Provide streams network changes to the client	61
5.7.2	Use Case ST-5.2: Client runs a provisioning use case ([LF TR-547] UC1x)	61
5.7.3	Use Case ST-5.3: Client runs the service deletion use case ([LF TR-547] UC1x)	61
5.8	Message Sequence example	62
5.9	Use cases beyond current release	66
5.10	Message approach (WebSocket example)	67
5.10.1	Basic interaction	67
5.10.2	Authorization example – WebSockets	67
5.10.3	Connecting to a stream example - WebSockets	68
6	Appendix	69
6.1	Appendix – Considering compacted logs	69
6.1.1	Essential characteristics of a compacted log	69
6.1.2	Order of events	69
6.1.3	Log segments	70
6.1.4	Partitions	70
6.1.5	Compaction in a real implementation	70
6.1.6	The Tombstone	70
6.2	Appendix – UML Model	71
6.3	Appendix – Stream record example	74
6.4	Appendix – Detectors, detected conditions and alarms	74
6.4.1	Detect	74
6.4.2	Detector	74
6.4.3	Condition	74
6.4.4	Condition detector	75
6.4.5	Detected condition	75

6.4.6	Alarm	75
6.4.7	Alarm detector	75
6.4.8	Traditional alarm reporting and legacy-properties	75
7	References	76
8	Definitions and Terminology	76
9	Individuals engaged	78
9.1	Editors	78
9.2	Contributors	78
10	Appendix: Changes between versions	78
10.1	Changes between v1.1 and v1.2	78

List of Figures

Figure 1	Example SDN architecture for WDM/OTN network	7
Figure 2	Yang: supported-stream-type	13
Figure 3	Yang: compacted-log-details	15
Figure 4	Yang: available-stream	16
Figure 5	Yang: log-record-header	24
Figure 6	Yang: log-record-body	26
Figure 7	Stylized view of example controller offering full compaction.	31
Figure 8	Stylized view of example controller offering emulated compaction	33
Figure 9	Yang: condition-detector (descriptions omitted)	37
Figure 10	Yang: alarm-detector and legacy-properties (descriptions omitted)	38
Figure 11	Hybrid Message Sequence Diagram for example implementation corresponding to Use Cases	64
Figure 12	Phases of interaction for Use Cases	65
Figure 13	Kafka compaction	69
Figure 14	Structure of the streaming model	71
Figure 15	Structure and content of the streaming model	72
Figure 16	Example of Augmentation of the LogRecordBody with some classes from the model	73

Document History

Version	Date	Description of Change
1.1	December 2021	Initial version of the Reference Implementation Agreement document on streaming for TAPI v2.1.3
1.2	December 2025	Updated to Linux Foundation document for TAPI v2.1.5

1 Introduction

1.1 General introduction

This LF Technical Recommendation (TR) is a supplement to the Reference Implementation for the TRANSPORT-API (TAPI) [LF TR-547].

1.2 Introduction to this document

The purpose of this document is to explain TAPI streaming and provide a set of guidelines and recommendations for use of TAPI streaming.

The target architecture for TAPI is provided in [LF TR-547]. The figure below is a copy of the figure provided in that document.

This document focuses on the autonomous flow of information via TAPI from SDN-C¹ to OSS/SDTN and from SDTN to OSS.

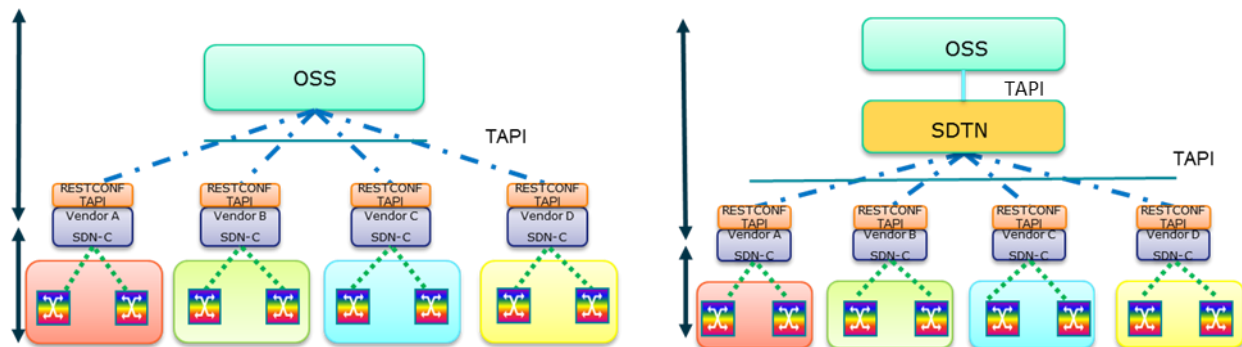


Figure 1 Example SDN architecture for WDM/OTN network

1.3 Specification

For TAPI, the Yang model files for a release are normative. It should be noted however, that some mandatory attributes are not shown as mandatory. In this case, the UML highlights which attributes are optional and which mandatory. This document indicates whether properties are mandatory, conditional mandatory or optional and in this respect this document is normative.

Behavioral aspects are not covered by the Yang model. This document covers behavioral aspects in section 5 Use Cases on page 41. Some uses cases are identified as normative and some informative.

Many parts of this document are simply informative and explanatory. Where this document is normative the words “normative” and/or “shall” will be used.

¹ See definitions in [LF TR-547].

2 Overview

2.1 Essential features and benefits

Streaming is the name for a type of mechanism that handles the providing of information from one system to another in some form of steady and continuous flow.

In the context of a Management-Control solution streaming is used primarily for the reporting (notification) of ongoing change of state of the controlled system from one Management-Control entity to another (usually superior) management-control entity. In this context as much of the information is derived from readings of instruments, the flow is often called telemetry².

The stream provides engineered flow such that peak load is averaged using some mechanism such as back-pressure and/or selective pruning of detail.

In the following discussion various terms such as controller, client and provider will be used. For the usage in this document of these terms see section 8 Definitions on page 76.

The definition allows for many alternative stream strategies using the same extensible structure. Streaming approaches are defined that:

- Focus on conveying TAPI entities, i.e., yang sub-trees (see 3.5 TAPI Application in detail on page 11).
- Provide an opportunity for event time reporting that is structured to allow for reporting of time uncertainty (see 4.8 Time on page 40).
- Allow a client (see section 8 Definitions on page 76 for usage of “client” in this document) to achieve and maintain eventual consistency with the state of the controlled system simply by connecting to the stream(s), i.e., with no need to retrieve³ current state prior to processing log reports (see 3.17 Eventual Consistency and Fidelity on page 29).
 - Makes it a provider (see section 8 Definitions on page 76 for usage of “provider” in this document) responsibility to send information to the client that ensures eventual consistency is achievable⁴, removing sequencing complexity.
 - Improves provide-side scale as simply based on a time-sequenced log of events as opposed to a combination of a time-sequenced log and a repository⁵.

Note: There is still a need for the client to “mark and sweep” to achieve full system realignment on major loss of communications.

- Allow for efficient recovery from temporary streaming channel communication failures (see 5.2.7 Use Case ST-0.7: Short loss of communications on page 48).
- Take pressure off of a client when under heavy load, recognizing that loss of some detail when under extreme pressure is inevitable and tolerable (see 5.2.7 Use Case ST-0.7: Short loss of communications on page 48).
- Remove the need for complex subscription in normal controller-controller interaction.
- Are structured to support, in future versions of this specification (see 3.9.2 Future combination considerations (by example) on page 21), the:
 - Providing of multiple alternative views of the controlled solution⁶

² This term loses relevance once the readings have been processed and abstracted but is often still used.

³ For example, using RESTConf GET.

⁴ Clearly the client implementation has to take advantage of this correctly as defined in the relevant use cases.

⁵ This removes the need to grab a snapshot of the entire repository to service a full-alignment get and removes the need for an alternative get fragment approach

⁶ It is assumed here that this would be performed through control of the scope of the context

- Creation of streams for temporary flows of information such as one or more spotlights on detailed state change or snapshots of current state

The TAPI streaming approach is aligned in principle with the approach taken by the [GNMI] community described in the [GNMI-SPEC].

2.2 TAPI application

TAPI Streaming can be used in several different applications. The primary application is one where a provider is offering an ongoing flow of state updates to a client, as depicted in Figure 1 above.

In this application the following assumptions apply:

- The client has one or more internal representations of the semantics (models) of the controlled system (network etc.). A representation may:
 - Relate to a subset of the TAPI model (e.g., just physical inventory)
 - Compress or expand parts of the model (e.g., Topology and Node are combined into a ForwardingDomain)
 - Be enriched with associations (e.g., some or all of the one-way navigations are converted to two-way navigations)
- The client maintains (stores in some form of repository) an ongoing live view of the state of the instances of elements in the controlled system so as to populate each of its representational forms
 - A mechanism is available that enables the on-going reporting, from the provider to the client, of change in information known to the provider.
 - Note: A view that is constructed from the currently known state will necessarily be plesiochronous⁷ with respect to the actual network state because of differential network and processing delays. After some period, the inaccuracies can mainly be corrected in a view of a particular past time such that the state that was present at some appropriately past time is determinable.
- When connected for the first time, the client must gain knowledge of current state prior to receiving information on change (changes alone are insufficient to provide a clear view of the system state especially recognizing that most states change very rarely – waiting for a change to determine current state is not viable).
 - On connection to the provider, the client gains alignment with the current state and then maintains alignment as the state changes
 - Through the on-going process the client populates its repository as appropriate and deals with the challenges of asynchronous receipt (e.g., the referencing entity arrives before the referenced entity)

Consequently, the provider aims to optimize the process of maintaining alignment for the client.

Note that TAPI Streaming is not currently intended to directly support:

- A client requiring access to the provider database supporting random queries
- A GUI

See section 5.9 Use cases beyond current release on page 66.

⁷ A plesiochronous system is one where different parts of the system are almost, but not quite, perfectly synchronised.

3 Summary of key considerations

3.1 Overview

This section examines the TAPI streaming capability in detail. Examples of UML and Yang are provided as appropriate.

The characteristics of streaming are described in general and are illustrated using example focusing on alarm reporting.

3.2 Existing RESTCONF notification mechanism (described in [LF TR-547])

RESTCONF specifies a traditional form of notification where the assumption is that a relatively short⁸ queue of notifications will be available on the provider to allow the client to get recent changes⁹ and alignment with current state (in the case of alarms, alignment with current active alarms) will be achieved by retrieving appropriate information from the TAPI context, e.g., using GET operations via RESTCONF.

3.3 TAPI Streaming available in 2.1.3

An Event source/server streaming mechanism is made available as an alternative to traditional notifications.

The streaming capability is distinct from TAPI Notification and is designed to better deal with scale and to provide an improved operational approach.

The method defined offers a “compacted log” (see 3.8.1.1 Compacted log on page 17) capability that allows the client to gain and maintain alignment with current state from the stream alone (with no need to get current state). The client can achieve eventual consistency (see 3.17 Eventual Consistency and Fidelity on page 29) by simply connecting to the relevant streams. The client will receive an ongoing view of change, assuming that the client is keeping up reasonably with the stream. The stream is designed to allow for some client delay with no loss of information fidelity.

When the client has a significant delay, there will be a loss of fidelity, due to compaction (see 3.8.1 Log storage strategy on page 17), but no impact on eventual consistency. If the client has a very large delay¹⁰, then a resync will be initiated by the provider. Resynchronization will be achieved simply by the client reconnecting to the stream from offset zero. This will again allow the client to achieve eventual consistency.

The streaming capability provides a reliable pipeline for reporting of change¹¹. This improves the information flow integrity and reduces the need for recovery and resynchronization.

3.4 Stream content

The streaming approach is generally applicable to all information available over TAPI from the provider to client.

⁸ Sufficient to buffer against variable client performance.

⁹ It will also allow the client to request changes starting from a specified sequence number.

¹⁰ The provider controls feeding the stream per client based upon backpressure from the client and is aware where it is reading from in the log, if the log record read is older than the tombstone retention (see definition/explanation later in this document), then the client will have potentially lost relevant tombstones and hence has possibly lost “eventual consistency”

¹¹ The capability assumes reliable communications such as TCP.

The streaming capability also offers an improved alarm structure (focusing on fundamental properties of the alarm and relegating legacy fields).

3.5 TAPI Application in detail

A management-control system, such as an Orchestrator, high level controller, OSS etc., has the role of configuring and adjusting the controlled system (network) to achieve intended capability (intent, service etc.). By monitoring and processing information (e.g., alarms) from the controlled system, the overall assembly of management-control systems can determine actions necessary to enable ongoing support of intent/service. The management-control systems can also identify repair action prioritization (via analysis of problems).

Management-control system components use TAPI to acquire, from the subordinate systems, information from a fragment of the overall network, e.g., the devices controlled by a controller, where that information is presented in terms of TAPI entities¹² within a TAPI Context.

The client maintains history and live views of the state of the things in the network so as to do the necessary analysis, hence that management-control system uses a mechanism providing autonomous updates and need NOT query the provider for states.

The overall solution is expected to have the following characteristics for the provider:

- Few (~2) direct clients¹³
 - Single OSS/orchestrator with several separate internal systems (Fault, provisioning, equipment inventory) and potentially some form of resilience
 - It is expected that TAPI is used at a point low in the management-control hierarchy close to the controlled devices (see Figure 1):
 - Interfaces above the OSS are unlikely to use TAPI such as:
 - Interface to customer management solutions
 - Interfaces direct to end user
 - At this point it is likely that the solution is somewhat traditional in nature with an OSS, or Orchestrator, or potentially an orchestrator and OSS operating in conjunction
 - It is assumed that the OSS/Orchestrator will be composed of functionally focused components (e.g., fault analysis, path computation) but that it will provide a relatively unified interface to the subordinate systems
 - The OSS/Orchestrator may be operating some form of resilience
 - Allows for a provider to divide up the information based upon entity type
 - This allows simple separation of topology from equipment from alarms
 - This simple split probably matches the normal gross partition of roles in an OSS/Orchestrator
 - On this basis it is assumed that there will be one or two clients for each stream type (perhaps up to 4 if there is both an orchestrator and an OSS which are both providing some alarm capability)
- Long-lived clients:
 - Clients remain “connected” for a very long time and if the connection is dropped the same client will usually try to reconnect

¹² A TAPI entity in this context is a Global Class, i.e., an instance identified by a uuid and hence the corresponding Yang sub-tree.

¹³ "Few (~2) direct client", is intended as order of magnitude, i.e. are not expected several tens of clients. In a future version there will be a broader consideration regarding client multiplicity for other applications

- Again, because of the point of use of TAPI in the management-control hierarchy and the role and purpose of the clients (OSS/Orchestrator), it is expected that the clients will be permanently connected.
- Provider maintains alignment with underlying system
 - The TAPI realization assumes a reliable input that ensures eventual consistency with current network state
 - As the client is an OSS/Orchestrator, it will have a repository.
 - The normal mode of operation is to align the repository with the view provided by the underlying system and to build a broader view of the network by integrating the views from many underlying systems.
 - For the OSS/Orchestrator to perform it is necessary to maintain alignment.

The primary focus for Streaming in TAPI 2.1.3 is simple and efficient ongoing alignment of a client with a view presented by a provider.

3.6 Summary of Streaming Characteristics

The key characteristics of the TAPI Streaming solution:

1. Ensures “eventual consistency” of the client with the view presented by the provider
 - Essentially, if the controlled system stops changing, once the whole stream has been received and processed in order by the client, the client view will be aligned with the corresponding controlled system state (assuming communication with all components in the controlled system)
2. Is built on a provider log of records recording change
 - The log is designed to enable “eventual consistency”
3. Guarantees delivery of each log record “at least once”
 - Clearly, this guarantee applies within a particular operational envelope as defined in this document
 - May deliver some information more than once, but this will be in a pattern that ensures “eventual consistency”
4. Is highly scalable and available
 - Boundless scale (with corresponding system resources)
5. Is highly reliable (network fault tolerant)
 - Provides an inherent high availability solution (assuming necessary implementation can be realized on a resilient server)
 - Is tolerant to network communications disruption allowing the client to resume from where it last successfully received a record.
 - Can feed multiple instances of client
6. Has low latency and high throughput on big data scale
 - Assuming the appropriate implementation technology
7. Offers flexibility in the division of information across streams
 - There can be multiple streams offered by a provider to a client where each stream differs from the others in terms of information content and/or protocol
 - In the case where there are multiple streams offered, the client may need to connect to several streams to get all the information it needs
8. Allows the client to re-consume records from a given stream any time.
9. Supports back-pressure¹⁴ from client to enable a reactive producer.

¹⁴ Applying some control to reduce the flow from the provider such that the client does not lose information.

3.7 Supported and available streams

The interface can offer many streams for a context. The client can determine, using calls on the provider, both the types of stream supported and the available streams that are active for connection and streaming. A variety of connection protocol, content, record strategy and storage strategy combinations might be offered. Clearly, some combinations will not be useful.

The next sections provide some fragments of Yang. The formal Yang deliverable shall be used as the definitive source of information on the encoding. This section indicates where properties are mandatory or optional.

3.7.1 Supported stream type

This structure allows the provider to report the streams that it can support, regardless of whether they are active or not.

Note that “record-retention” and “segment-size” (see section 6.1 Appendix – Considering compacted logs on page 69) are both string fields. They both have potential for complex structuring and may require future formalization. For example, “record-retention” is either time or capacity and also has a key word when the retention is “FOREVER”. In a future release this may become a complex structure.

```

grouping supported-stream-type {
  leaf stream-type-name {
    type string;
    config false;
    description "Name of the stream type.";
  }
  leaf record-retention {
    type string;
    config false;
    description "Time in minutes.
      Statement of retention time and/or retention capacity in bytes.
      Key word 'FOREVER' means that records will never be removed from the log.
      May be overridden for particular cases of specific LogStorageStrategy (via augment).";
  }
  leaf segment-size {
    type string;
    config false;
    description "Size of sub-structuring of the log.";
  }
  leaf-list record-content {
    type object-class-identifier;
    description "Identifies the classes that are supported through the stream.
      The list may be a subset of the classes within the context.";
  }
  leaf log-storage-strategy {
    type log-storage-strategy;
    config false;
    description "Indicates the storage characteristics of the log supporting the stream.";
  }
  leaf log-record-strategy {
    type log-record-strategy;
    config false;
    description "Indicates the type of content of each log record.";
  }
  uses tapi-common:global-class;
  description "Definition of a supported stream type.";
}

```

Figure 2 Yang: supported-stream-type

For this structure, a solution shall provide the following support:

- Mandatory:
 - stream-type-name
 - record-retention
 - record-content
 - log-storage-strategy
 - log-record-strategy
- Optional
 - Segment-size

Considering the supported-stream-type yang, using the “record-content” leaf, the provider can indicate the entity types supported by a stream, the storage and record strategies.

The segment-size is a free choice made by the provider depending upon system engineering. Larger segments may interfere with compaction delay where the stream is handling entities with a low rate of change whereas smaller segments will require additional segment creation activity.

Information may be divided into separate streams. There is no restriction on choice of division of the information into streams. A provider could choose to have a stream per class or to have streams that aggregate classes together that have similar lifecycles etc. It should be noted that for TAPI 2.1.3 ALL instances in the context of any object-class-identifier listed in record-content will be streamed, i.e., there is (intentionally) no client control of filtering¹⁵.

The supported-stream-type is also augmented with connection-protocol-details which provides a list of allowed-connection-protocols in freeform string (to be normalized through agreement). Candidate protocols include “websockets” [RFC6455] and “sse” [W3C SSE].

For a non-compacted log, record-retention is a free choices made by the provider depending upon system engineering. Clearly, the longer the retention, the larger the log.

For a compacted log, record-retention time should be “FOREVER”.

The supported-stream-type can be augmented with compacted-log-details which provides additional parameters for compacted log applications. This augmentation shall be applied when the solution is running compacted logs.

The description in this document focusses on Compacted logs (some other log strategies are also considered). Compacted logs are explained at various points in this document. The key is as noted in the Yang description below, which is essentially that the log holds only the latest record about each entity. Once a such entity is deleted a “Tombstone” log record (see section 6.1.6 The Tombstone on page 70) for the entity is appended to the log (which then becomes the latest record for the thing). The tombstones are only held for a relatively short period¹⁶ of time (the tombstone-retention).

¹⁵ It is assumed that the Context has been designed to include all the entities that the client is interested in and hence the process of Context formation at the provider does all the necessary filtering to ensure that the client gets what it needs.

¹⁶ The time is determined by system engineering considerations. For example, in a specific solution, comms failures between the client and the provider of greater than 4 hours may be considered as extremely unlikely, and general system recovery from a 4 hour log may take 2 hours. In this case a tombstone retention of slightly more than 6 hours is suitable. This is a relatively short period of time.

```

grouping compacted-log-details {
  leaf tombstone-retention {
    type string;
    config false;
    description "Time in minutes.
      The time period for which a Tombstone record will be held in the log from when it was logged.
      This provides an adjustment to the essential Compaction strategy such that after the tombstoneRetention period there will be no
      records about a particular thing that existed but no longer exists.
      Tombstone retention overrides recordRetention for Tombstones.
      Key word 'FOREVER' means that Tombstone records will never be removed from the log.
      Can be adjusted by an administrator (via a separate view) through the life of the stream.";
  }
  leaf compaction-delay {
    type string;
    config false;
    description "Time in minutes.
      The delay between logging the record and making the record available for compaction.
      This provides an adjustment to the essential Compaction strategy such that there may be several distinct records for the same thing
      in the where those records are not older than the Compaction Delay.
      Can be adjusted by an administrator (via a separate view) through the life of the stream.";
  }
  description "Details relevant for a CompactedLog.
    The essential Compacted Log strategy is to remove historic records about a particular thing such that only the latest record about each
    thing exists in the log.
    The essential strategy is refined by the parameters of this structure.";
}

```

Figure 3 Yang: compacted-log-details

For this structure, a solution shall provide the following support:

- Mandatory:
 - tombstone-retention
 - compaction-delay

For the compacted log, the supported-stream-type information is augmented with compacted-log-details that includes tombstone-retention¹⁷ (essentially retention of records about deleted events – see 3.8.1 Log storage strategy on page 17) and compaction-delay settings. Both are free choices made by the provider depending upon system engineering.

¹⁷ A Tombstone is a record that provides sufficient to express the delete of an entity, tombstone-retention is the time for which a Tombstone record will be held in the log.

3.7.2 Available Streams

This structure allows the provider to report the streams that are currently available.

```

grouping available-stream {
  leaf connection-address {
    type string;
    config false;
    description "Provides the address for the connection.
      The format of the address and attachment mechanism will depend on the connection protocol defined in another attribute of this
class.";
  }
  leaf stream-state {
    type stream-state;
    config false;
    description "The state of the stream.";
  }
  container supported-stream-type {
    uses supported-stream-type-ref;
    config false;
    description "Identifies the type of stream that is available for connection.";
  }
  leaf stream-id {
    type string;
    config false;
    description "The id of the stream.";
  }
  leaf connection-protocol {
    type string;
    config false;
    description "Names the connection protocol for this particular available stream.
      The connection protocol is chosen from the list of connection protocols identified in the referenced SupportedStreamType.";
  }
  uses tapi-common:global-class;
  description "Details of a stream that can be connected to by a client application.";
}

```

Figure 4 Yang: available-stream

For this structure, a solution shall provide the following support:

- **Mandatory:**
 - **connection-address:** the location of the stream
 - The address structure and connection method will depend upon the connection-protocol
 - **stream-state:** indicates whether the stream will deliver records to the client or not
 - **supported-stream-type:** references the description of a stream supported by the provider
 - **stream-id:** id of the stream
 - **connection-protocol:** the protocol for this particular stream instance chosen from the list of available protocols for the stream type

3.8 Streaming approach and log strategy

The streaming solution assumes that the provider is delivering information in sequence from a log. In TAPI 2.1.3 a log approach oriented towards maintaining alignment is provided. The stream mechanism defined allows for different log strategies¹⁸ along with corresponding augmentation.

¹⁸ These are described in the Yang/UML.

3.8.1 Log storage strategy

There are four log-storage-strategy provided: COMPACTED, TRUNCATED, FULL-HISTORY and FULL_HISTORY_WITH_PERIODIC_BASELINE, these are described in the subsections below.

3.8.1.1 Compacted log

An implementation conforming to this specification **MUST** support the COMPACTED log-storage-strategy (the characteristics of this mechanism are described in 6.1.1 Essential characteristics of a compacted log on page 69). This log-storage-strategy enables the client to achieve and maintain eventual consistency without the need to get current state.

Not all change statements (whole entity statements) related to an entity are retained. Older changes are intentionally pruned out of the log in favor of newer changes.

- The log will have the latest record related to each entity that exists in the controlled system.
 - This enables achievement of eventual consistency.
- Some additional recent changes will also be present in the log as compaction is intentionally delayed.
 - This enables a delayed client to catch back up with no loss of fidelity.
- Records that are not the latest for an entity and that are older than the compaction delay time (i.e., the “Cleaner Point” as shown in Figure 13 Kafka compaction on page 69) will be removed.
 - This allows an overloaded client to maintain a view of non-fleeting changes whilst suffering an acceptable loss of fidelity where there is high intermittency.
 - This allows a new client to align without having to receive large volumes of uninteresting history.

When an entity is deleted, a Tombstone record is added to the log for that entity. The Tombstones are held for Tombstone retention:

- Compaction, after the compaction delay, will remove all but that tombstone record.
- Tombstones are removed once they have persisted for the tombstone retention period (i.e., the “Deletion Retention Point” as shown in Figure 13 Kafka compaction on page 69)
 - Note that without this special tombstone retention behavior, the log growth would be unbounded.

3.8.1.2 Truncated log

The log holds records about all recent changes relevant to the target content for the log as defined by record-content in the supported-stream-type definition. Truncation will occur as a result of volume of records, age of records or some other criteria. Hence, the log content is limited either by log size or by content age.

- For a size bounded log, once the log has reached its content limit, as each new record arrives, space is cleared, to allow the new record to be added, by deleting the oldest records.
- For an age bounded log, records older than the age limit are deleted.

This can be considered as a traditional notification queue where recent records have been retained. This log-storage-strategy does not allow the client to achieve eventual consistency without getting the current state.

See 3.9.2 Future combination considerations (by example) on page 21 for potential uses. Some uses may require augmentation of the supported-stream-type structure to allow full definition. Possible augments will be defined in a future version.

3.8.1.3 Full history log

The log holds all records about changes relevant to the target content for the log as defined by record-content in the supported-stream-type definition since the context was created. Unlike the compacted and truncated cases, this is essentially a boundless log holding the entire history.

See 3.9.2 Future combination considerations (by example) on page 21 for potential uses. Some uses may require augmentation of the supported-stream-type structure to allow full definition. Possible augments will be defined in a future version.

3.8.1.4 Full history with periodic baseline log

The log holds all records about changes relevant to the target content for the log as defined by record-content in the supported-stream-type definition since the context was created and also provides periodic baselines¹⁹ within the stream that allow the client to interpret recent changes without needing to review the entire history. The form of the baseline records and settings for the log have not been defined in TAPI 2.1.3. The application of this type of log is for further study.

See 3.9.2 Future combination considerations (by example) on page 21 for potential uses. Some uses may require augmentation of the supported-stream-type structure to allow full definition. Possible augments will be defined in a future version.

3.8.2 Log record strategy

There are three log-record-strategy, `WHOLE_ENTITY_ON_CHANGE`, `CHANGE_ONLY`, `WHOLE_ENTITY_PERIODIC`, these are described in the subsections below.

3.8.2.1 Whole entity on change

The log-record-strategy fully available in 2.1.3 is `WHOLE_ENTITY_ON_CHANGE`. The combination of this log-record-strategy used in conjunction with log-storage-strategy of `COMPACTED` is described in detail in this document to enable the client to achieve eventual consistency.

In this case, each record in the log holds a full copy of the entity. A full copy of the entity is stored when the entity is created and when any state changes, i.e., the solution logs (stores) a full representation of the entity with current values after each change, not just the changed property. Hence, the whole entity that has the changed property is streamed. The specific changes can be identified by comparing the current record with the previous record. Note that the entity could have more than one changed property.

The model is such that:

- Each property that changes regularly is isolated in its own dedicated small class²⁰
 - E.g., The alarm (detector) is considered as a class. Alarms are isolated from configuration data for the related entity and from each other.
- Large data structures that are invariant or change rarely can be grouped in composite classes.

¹⁹ A baseline is a statement of absolute state (in this case of all resources relevant to the stream). Updates alone cannot be meaningfully interpreted as they are updates against some state. A full history will have all state changes, but starting from the first moment in the history to determine the current state, running through all changes would be extremely expensive. Hence providing a baseline from time to time can speed up alignment when a system connects for the first time.

²⁰ Currently `OperationalState` is part of the main entity where there is a large volume of slow changing data. Operational state actually has an alarm like behaviour and should be reported as an alarm.

- E.g., The CEP where Configuration data (both intent and actual) is collected into a single class. The data in an instance changes rarely.
- For optimum support of change of properties, the small property class reference the configuration items that they relate to and NOT the reverse (i.e., be a decoration, e.g., an alarm references the CEP or MEP)

This record strategy can be used with any of the storage strategies.

3.8.2.2 Change only

This provides a skeleton of the instance of the class (identifiers and basic structure) with only the changed property (or properties) present.

IMPORTANT NOTE: This approach has not been fully developed. It is possible to use this log-record-strategy in conjunction with log-storage-strategy of COMPACTED to enable the client to achieve eventual consistency.

This record strategy can be used with any of the storage strategies, but it requires specific behaviour for the COMPACTED log storage strategy:

- The entity-key (see 3.10 Record content on page 23) for the change records must be distinct from the entity-key for whole entity recorded at create such that compaction does not remove the base entity. The key could reasonably be the uuid of the entity concatenated with the appropriate sequence of local ids for the attribute (i.e., the tree).
- The Delete and Tombstone behavior must be such as to Compact out (and hence remove) all elements related to the subtree under the parent record when the parent is deleted.
- Properties can be added and deleted using subtree tombstones.

See 3.9.2 Future combination considerations (by example) on page 19 for potential uses. Some uses may require augmentation of the supported-stream-type structure to allow full definition. Possible augments will be defined in a future version.

3.8.2.3 Whole entity periodic

Using this strategy, the information is streamed on some periodic basis even if it has not changed. This approach has not been fully developed. See 3.9.2 Future combination considerations (by example) on page 19 for potential uses. Some uses may require augmentation of the supported-stream-type structure to allow full definition. Possible augments will be defined in a future version. This log-record-strategy could be used to enable the client to achieve eventual consistency.

3.9 Using the stream

3.9.1 Streaming the context

The primary application of streaming in TAPI 2.1.3 is in a solution where a client needs to gain and maintain alignment with a context presented by a provider:

- The provider presents a view in terms of a context and all of its contained instances.
- The client maintains alignment with that view.
- The stream is used for gaining and maintaining alignment with a view, i.e., a TAPI context.

The next subsections consider the client connection to and receiving from a stream. The approach is described for the case where the provider offers COMPACTED and hence

WHOLE_ENTITY_ON_CHANGE. The description assumes that the client is consuming the stream at a far greater rate than the stream is being filled. The following provides a brief sketch of alignment. The process is discussed in far greater detail later in the document (see 5 Use Cases on page 41).

3.9.1.1 Effect of streaming approach and compacted log characteristics on alignment

There are two distinct aspects of alignment:

- **Absolute State:**
 - **Eventual Consistency:** Ensures that the client view of state of controlled system aligns with the state of the view of the actual controlled system as presented by the provider.
 - If the controlled system stops changing, once the stream (all changes) has been absorbed by the controller, its view of the current state of the system will be aligned with the actual state of the system.
 - **Context Detail:** The information agreed to be conveyed from the provider to the client.
 - Note that some clients will choose to selectively prune out information that is not relevant to them.
 - Information conveyed in a context may be temporarily increased as a result of a recognized short-term need (value)
- **Change of state:**
 - Detail will necessarily be lost (loss of fidelity) when there are communications failures, but the loss will be such that less relevant information is lost first.
 - Removal of noise (such as rapid clearing and then re-raising alarms) will be generally beneficial.

3.9.1.2 Preparing to connect

Once the client has identified the available streams to connect to, the client simply acquires the necessary authorization (see 5.10 Message approach (WebSocket example) on page 67 and 5.8 Message Sequence on page 62) and connects.

3.9.1.3 Initial connection

On initial connection, the client provides a null token. This causes the provider to stream from the oldest record. The client can continue to consume records from the stream ongoing.

The initial records received by the client will be for the entities that have not changed for a “long” time²¹.

3.9.1.4 Tombstone (Delete) retention passed

As the client continues to consume the stream it progresses past the Tombstone (delete) retention point, i.e., is receiving records that have a timestamp that is less than the Tombstone retention (delay) from the current time (see 3.8.1 Log storage strategy on page 17 and 6.1.1 Essential characteristics of a compacted log on page 69 for a brief explanation of the log structure), and recent tombstones will be received along with newer changes²².

Compaction will have removed multiple reports about the same entity, but as the stream progresses further it is possible that an update is received that overwrites previously received entity state or a tombstone is received that deletes an entity that was read earlier. This is where compaction had not yet removed the entity when the stream was started (potentially because the event causing the newer record had not yet occurred).

3.9.1.5 Compaction delay passed

After some time, the client consumes past the compaction delay point (i.e., is receiving records that have a timestamp that is less than the compaction delay from the current time). From this point onwards the client

²¹ For example, if the system has been running for three years and a thing was created when the system started. If that thing has never changed since creation, then the record of its creation from three years ago will still be in the log.

²² Tombstones (deletes) are only retained for a limited time. Tombstone records older than the Tombstone retention are removed from the log.

is receiving all recent changes and is aligned with network state as it was perceived by the provider at some recent point in time. Whilst receiving records that are newer than the compaction delay point the client will receive all event reports for the context (see 3.8.1 Log storage strategy on page 17 and 6.1.1 Essential characteristics of a compacted log on page 69 for a brief explanation of the log structure).

3.9.1.6 (Eventual) Consistency achieved

If the controlled system stopped changing, then the client would eventually reach the newest record and would be aligned with the provider view of the state of the controlled system²³.

3.9.1.7 Degraded performance

Information fidelity is reduced if the client slips back by more than the compaction delay as compaction will remove some change detail.

3.9.1.8 Need for realignment

If the client processing of a stream is delayed by more than the tombstone retention²⁴ such that the backpressure on the stream takes the provide log read point to a record older than the tombstone retention time, the provider will cause the connection to drop.

When the client detects this, it will reconnect, normally with the token for the most recently processed record from the stream. The provider will recognize this as for a record older than the tombstone retention and will stream from offset zero (i.e., the oldest record in the log). This will cause the client to enter into full realignment.

The behavior of the provider at this point is equivalent to that when there is an initial connection. The client may use a “mark and sweep” strategy to minimize the disruption to its view of current state.

3.9.1.9 Summary

The above detail can be summarized:

1. The client will connect for the first time and the provider will stream from the oldest record in the log.
2. Where the client loses communications or loses a record for some other reason, it can reconnect to the provider indicating the last record that it successfully received. Missed records are then streamed as appropriate where tombstone retention has not been exceeded.
3. Where the client has crashed it connects to the stream as if for the first time in 1 above
4. Where tombstone retention has been exceeded the provider takes the client back to the start of the stream
 - Compaction removes noisy history and allows rapid alignment with current state through a “replay” of the compacted history.

3.9.2 Future combination considerations (by example)

The choice of the protocol for streaming is in principle independent from the stream characteristics, although the protocol chosen must match the stream reliability needs. There are several potential applications beyond those documented this first release.

3.9.2.1 Many clients

Although, at this stage, all TAPI applications appear to have a small number of clients, it is possible that applications will emerge with many clients for a single type of information. stream of information.

For some “many clients” application a combination of log-storage-strategy = TRUNCATED and log-record-strategy = WHOLE_ENTITY_PERIODIC would appear to be suitable. In this case the TAPI provider may

²³ Because the provider logs the whole entity on each change then the most recent record for an entity, retained after compaction has removed earlier records, will include all of its properties.

²⁴ The TCP buffering will provide some additional time for a client beyond the tombstone retention.

supply the clients directly or via an intermediate solution to deal with load. It is also possible that the TAPI provider might stream full content and then the intermediate solution might support a subscription method.

3.9.2.2 Many views (and many clients, a few per view)

In this case, the solution would appear to be one with multiple contexts, one per view. Here any relevant log-record-strategy and log-storage-strategy could be supported. Multiple contexts would benefit from support of further features such as building context (discussed briefly in section 5.9 Use cases beyond current release on page 66).

It is assumed that:

- Each context will appear as a separate independent instance of a TAPI sever.
 - Each context will have its own dedicated streams.
- A context will expose a subset/subgraph of the overall network where that subset/subgraph/ may be a small compared to the overall network.
- The identifiers in each context
 - May be different for the same underlying entity, to improve security of access where views are being offered to distinct and independent clients.
 - May be the same for a related entity across two or more contexts, to allow for partitioned views between related clients. In this case it is expected that the clients will be sharing data. In a basic example, one client may deal with equipment whilst another deals with network topology. The model the equipment client supports has references to the model the network client supports.

3.9.2.3 Many short-lived clients

In this case, where each client samples the information from some combination of streams for a very short period. This appears to be some combination of many views and many clients.

3.9.2.4 Live measurements

Where there is a low number of clients, each requiring a reasonably up-to-date view including values related to measurements (e.g., delay, temperature, power) which may:

- tend to dither around a value or tends to increase on an ongoing basis (or decrease on an ongoing basis), a combination of log-storage-strategy = COMPACTED and log-record-strategy = WHOLE_ENTITY_PERIODIC would appear to be suitable.
- tend to change rarely, a combination of log-storage-strategy = COMPACTED and log-record-strategy = WHOLE_ENTITY_ON_CHANGE would appear to be suitable

3.9.2.5 Threshold Crossing

This depends upon the behavior of the threshold source. For a source that:

- reports threshold crossing at some point within a period of measurement, at the end of the measurement period resets the threshold without a clear and then potentially reports the threshold crossing in the next period etc., the solution would best use log-storage-strategy = TRUNCATED.
- reports threshold crossing and recovery over a continuous measurement, the solution would best be the same as the alarm solution, i.e., log-storage-strategy = COMPACTED and log-record-strategy = WHOLE_ENTITY_ON_CHANGE

3.9.2.6 Periodic measurement data

Where a measurement is taken over a period of time and where the measurement is repeated regularly (potentially over adjacent periods of time). An example of this is the 15-minute Performance measurement.

- At the end of the measurement period the result of measurement is streamed

- The natural approach would be to use Log-storage-strategy: TRUNCATED and log-record-strategy: WHOLE_ENTITY_PERIODIC
 - This provides a short history to handle communications problems.

Some periodic measurements have predictable normal characteristics.

- Consider an error measurement on a photonic system.
 - This is normally zero and hence the errored second count in a 15-minute period is normally zero as is the severely errored second count. On that basis indicating which detectors are enabled (via a compacted log with appropriate compaction delay to ensure an interpretable short history and reporting only the non-zero counts can lead to significant efficiency gains.
 - In this case the measurements use a log with log-storage-strategy: TRUNCATED and log-record-strategy: WHOLE_ENTITY_PERIODIC
- Consider a power measurement on a photonic system averaged over a 15-minute period.
 - This normally changes slowly such that a sequence of measurements will have the same value. On this basis reporting an initial value and subsequently only changes lead to significant efficiency gain
 - In this case the measurements use a log with log-storage-strategy: COMPACTED and log-record-strategy: WHOLE_ENTITY_ON_CHANGE where the change is considered from one measurement period completion to the next

3.9.2.7 Bulk Performance Monitoring (PM) data

Essentially covered using multiple periodic measurement response where the current-data instance is streamed once the granularity period ends and the current-data is rolled to history-data (i.e., streaming of history-data creation)²⁵.

3.10 Record content

The stream-record allows for multiple log-records each of which includes a log-record-header and optionally a log-record-body.

The log-record-header provides information common to all records.

The Tombstone record may have only a header.

²⁵ Further work is required in the area of PM collection and PM streaming.

The log-record-header is as below.

```

grouping log-record-header {
  leaf tapi-context {
    type tapi-common:uuid;
    config false;
    description "The identifier of the context.";
  }
  leaf token {
    type string;
    config false;
    description "A coded (and compact) form of the fullLogRecordOffsetId.
      This property is used to request streaming from a particular point (e.g., the last correctly handled record).";
  }
  list full-log-record-offset-id {
    key 'value-name';
    config false;
    min-elements 1;
    uses tapi-common:name-and-value;
    description "In a complex log solution there may be various parts to the log.
      The record token is a compressed form of log record reference.
      This property provides the verbose form
      For example, it may include:
      - stream id
      - topic
      - partition
      - partition offset
      - sequence number (the offset is essentially the sequence number associated with the partition)";
  }
  leaf log-append-time-stamp {
    type tapi-common:date-and-time;
    config false;
    description "The time when the record was appended to the log.";
  }
  leaf entity-key {
    type string;
    config false;
    description "The identifier of the entity that is used in a Compacted log as the compaction key.
      The entityKey value, where appropriate, may be based upon the identifiers from the event source.
      It can be built from some specific detail combination that meets the necessary uniqueness and durability requirements.
      entityKey is the value used during compaction.
      Ideally it is a UUID format, if this can be formed from the source identifier.";
  }
  leaf record-type {
    type record-type;
    config false;
    description "The type of the record.
      Can be used to understand which elements of the record will be present.";
  }
  leaf record-authenticity-token {
    type string;
    config false;
    description "none";
  }
  description "The header of the log record providing general parameters of the record common to all records.";
}

```

Figure 5 Yang: log-record-header

For this structure, a solution shall provide the following support:

- **Mandatory:**
 - token: The normal identifier of the record.
 - After connection failure, it is this value that is used by the client to indicate to the provider the last processed record such that the provider can determine which record to send next.

- log-append-time-stamp: The timestamp for the record being placed in the log
- entity-key: the reliable identifier for the entity that is used for compaction and tombstone processing.
 - This need not be the entity uuid so long as it is invariant for the life of the entity, and it is unique.
- record-type: Indicates the type of record (e.g., if the record is Tombstone)
- Conditional Mandatory:
 - full-log-record-offset-id: the long hand identifier of the record to allow the client to interpret stream structure and position
 - Mandatory if the token is not suitable to confirm sequence integrity
 - Provides suitable detail to identify sequence, e.g., where the underlying log is partitioned²⁶ such that there are multiple parallel sequences
- Optional
 - tapi-context: This field can be omitted for TAPI 2.1.3 as focus is a single context.
 - This uuid identifies the applicable TAPI context. For TAPI 2.1.3 the mapping between the context and its uuid is unspecified.
 - In future this will enable systems that support more than one context to ensure the context of the stream is present in the record.
 - record-authenticity-token: allows the provider to supply a value with each record, that is usually different from record to record (e.g., a digital signature where the value of which is set by combining some shared secret with the record content²⁷), that the client has a mechanism for confirming so as to validate that the record came from the expected originator and hence to protect against records being inserted in the stream from some unauthorized source.

The log record body includes generalized content and is augmented with specific content depending upon the value of a control property, record-content.

²⁶ Kafka, described very briefly in 6.1 Appendix – Considering compacted logs on page 38, supports partitioning of logs to improve scale and performance.

²⁷ The mechanism for generation of the value and validation of the value has not be set at this stage, it is intended that in a later TAPI version a formal approach to setting the value will be specified.

```

grouping log-record-body {
  container event-time-stamp {
    config false;
    uses approx-date-and-time;
    description "Time of the event at the origin of the event that triggered the generation of the record.
      The structure allows for time uncertainty.";
  }
  leaf event-source {
    type event-source;
    config false;
    description "Indicates whether the source is controlled (under management control) or potentially chaotic (under resource control).
      The time characteristic of the source may be determined from the metadata describing the resource (e.g., a detector).
      Where there is an alternative (and probably more detailed) source of information on time characteristic this attribute can be omitted.";
  }
  list additional-event-info {
    key 'value-name';
    config false;
    uses tapi-common:name-and-value;
    description "Addition information related to the event such as change reason where changeReason would be the name and the value
      text would provide information on the reason for change.";
  }
  leaf-list parent-address {
    type string;
    config false;
    description "Where the entity is a local class this provides the ordered list of ids from the closest global class (a UUID cast as a string)
      to the direct parent (which may be the global class).
      The field can include all entities back to the Context and hence can be used for global classes where the tree is being represented
      in full.
      Gives the position of the entity in the address tree (usually containment) that is raising the event by providing the name/id values in
      the address of the parent.
      Is the sequence of named levels in the tree up to but excluding the entity of the notification.
      It includes the device id where relevant.";
  }
  leaf record-content {
    type object-class-identifier;
    config false;
    description "The identifier of the object class in the record body detail.
      This property is used to control the conditional augmentation of the body with detail.";
  }
  description "The specific details of the Record.";
}

```

Figure 6 Yang: log-record-body

For this structure, a solution shall provide the following support:

- **Mandatory:**
 - event-time-stamp: The time of the event as recorded by the originator
 - record-content: Identifies the structure of the content and is used to control the augmentation.
 - Where the entity-key is a uuid and hence is sufficient alone to identify an entity that is to be deleted, the record-content for a DELETE record can be set to ANY_CLASS (and may have no augment).
- **Conditional Mandatory**
 - parent-address: Provides the yang tree location.
 - Mandatory for all cases other than events related condition-detector and where the record-content is DELETE
 - Example for a connection-end-point:

```

"parent-address" : [
  "topology-uuid:4e537278-79f8-39ad-804b-f0b553cb2ffb",
  "node-uuid:7f65a8c1-4e0d-3296-b06a-460b2367ca76",

```

```
"node-edge-point-uuid:e35a31b2-1096-38f9-8e3e-dca010e2f191"
],
```

- By providing the parent-address, the provider no longer needs to update the parent entity with the changes to the containment list property corresponding to the streamed entity, the relevant property in the tree can be updated by the client. This approach removes some race condition atomicity challenges
- In the example above the cep-list of the node-edge-point identified in the parent-address would be updated with the uuid of the connection-end-point
- Note: The condition-detectors itself does not have a parent and hence the field can be omitted in the case of a condition-detector as the measured-entity-uuid of the condition-detector (see 4.5 Condition detector and alarm structure on page 36) is sufficient to identify the affected entity.
- Note: Where the entity-key is a uuid and hence is sufficient alone to identify an entity that is to be deleted, the parent-address for a DELETE record can be omitted.
 - The onus is on the client to update the corresponding containment tree references, so in the example above a connection-end-point delete would require the client to remove the connection-end-point from the cep-list of the appropriate node-edge-point.
- Optional
 - event-source: Indicates the dynamic nature of the event.
 - additional-event-info

The log-record-body is augmented with an instance of a class. This allows for any class from the model to be reported. Hence, when using the Compacted Log approach, the streaming mechanism can be used to gain and maintain alignment with all entities in a context.

3.10.1 Considering parent-address

The parent-address enables a client to position an entity in the Yang tree.

- Where the entity is a Global class and hence has a UUID (can be considered as a Domain Driven Design [DDD] Aggregate), the parent address shall provide the position of the entity in the yang tree.
- Where the entity is a local class and hence has a local id relative to the UUID of its containing Global class, the parent address shall provide any necessary levels of nesting of local-id, the UUID of its Global class and then the yang tree.
- For alarms and events that are reported as independent entities, the parent-address may include address of the indirect parent (i.e., the entity against which the alarm is raised).

3.11 Considering order/sequence and cause/effect

3.11.1 Time

When determining the cause and effect of any behavior in the controlled system it is necessary to have visibility of relevant state and to know the time of each change of state. The time units must be sufficiently fine to allow all relevant event sequencing to be determined²⁸.

The time of change at the source of change needs to be propagated as data in the stream and hence needs to be in the report of each instance of the changed entity (and in any stored form that exists between the source and the stream). This is recorded in event-time-stamp.

²⁸ It may be beneficial to add an indication of time granularity to assist in cause/effect evaluation. For this to be fully beneficial the accuracy of synchronization of time would also need to be determined).

The time the record was logged is log-append-time-stamp.

3.11.2 Backend stream details

The implementation solution may partition the log supporting a stream (see section 6.1.4 Partitions on page 70). This may cause stream content reordering. Clearly, event reordering across different sources is a fundamental behavior due to relativity and differential delay. It is expected that the order for events from each single state machine will be maintained.

Regardless, the timestamp granularity must be sufficient to ensure relevant chronological order can be recovered. This also assumes that robust time synchronization mechanism is present at each event source and hence in the controller/controlled system as a whole.

3.12 The Context

TAPI is a machine-machine interface. As noted earlier, the client maintains a local repository representation of the information from the provider. The information is defined in terms of the context. As the context has been agreed (currently, up front during system design), the context is what the client wants/needs to see.

Where the client needs less detail than is provided by a specific context the client can:

- Locally filter out information that is not of interest.
- Change the context.
 - The context can be modified as necessary so long as other clients (and the provider) agree with the change.
- Request construction of a specific context
 - Several contexts can be provided.

On this basis:

- Individual specific queries on the provider are not necessary.
- Notification filters, that are not simply the realization of the view, are not necessary. The context defines the filtering for the client.

Any changes in the required information are handled by changes in the context. See later discussion on changing the context and spotlighting in section 5.9 Use cases beyond current release on page 66.

3.13 Handling changes in the Context

The stream relates to the entities in the context. Changes include:

- An instance of a thing being added/removed to/from context within the current definition of the context.
 - E.g., the creation of a connection
- The context definition being changed such that an instance of a thing appears/disappears.
- An instance of a thing in the context changing such that an element is added/removed from the thing.
- The value of a property of an instance of a thing already in the context changing.

See later discussion on changing the context in section 5.9 Use cases beyond current release on page 66.

3.14 Reporting change

For the compacted log solution, if the log-storage-strategy is `WHOLE_ENTITY_ON_CHANGE`, whole entities are streamed on creation, deletion and change of a property. Hence, for example, if a single property

in a CEP changes, the whole CEP is logged and streamed. The parent-address indicates where the CEP is in the tree and hence which NEP and node have changed (in CEP detail)²⁹.

3.15 Model implications

Separation of properties that have a slow rate of change (intent-driven config) from properties that have a high rate of change (network-derived state) and isolating the high rate of change properties in independent separate state entities is advisable. The alarm and state model provided for streaming allows a separation of small per-state entities from large slow changing config entities. The current TAPI OAM model also isolates properties related to monitoring results from properties related to configuration.

3.16 System engineering

For the solution to operate reliably:

- System engineering must be such that under normal and “normal bad” day circumstances the client is able to keep up well with the provider (otherwise the client will suffer ongoing alignment issues in terms of lag and potentially in terms of fidelity).
 - Eventual consistency is acceptable in a control solution if there is only a “short” delay to alignment with any specific state.
- For realignment to be successful, the client must be engineered to be able to read all records in the tail of the log (starting from the oldest) up to the Tombstone retention point in under the Tombstone retention time. On this basis
 - The tombstone retention time may differ for different cases and different streams.
 - For a stream with only limited volumes of long term records the tombstone retention could be quite short
 - Under these circumstances, tombstone retention is probably determined by likely communication failure duration.
 - For the case where alignment takes days, tombstone retention would need to be in terms of days.

The solution can be tuned to balance pace to achieve consistency (eventual consistency) with the fidelity of information when under stress³⁰.

3.17 Eventual Consistency and Fidelity

3.17.1 Eventual Consistency

A simple way to understand eventual consistency is to imagine a network that is changing such that there is a stream of changes and where the client has not absorbed the stream. If suddenly the network were to stop changing, then, once the client has absorbed the entire stream, the client will be aligned with network state.

3.17.2 Fidelity

In this document, fidelity is the degree of exactness with which the controlled system behavior is represented via TAPI streaming.

Loss of fidelity is the loss of some details of change (without losing eventual consistency). For example, the operational-state property of a CEP may initially be ENABLED, then become DISABLED and then

²⁹ Only the CEP needs to be notified as this identifies where the NEP and the node have changed.

³⁰ Compaction is used intelligently to reduce realignment time whilst minimizing probability of loss of detail.

ENABLED again. Loss of fidelity may lead to the operational-state being observed as continuously ENABLED.

3.17.3 Related stream characteristics

Considering the supported-stream-type structure discussed in 3.7.1 Supported stream type on page 13, there are two key time settings for the compacted log solution “tombstone-retention”³¹ and “compaction-delay”³².

- Client read delay is less than compaction delay.
 - Client is behind on absolute state but is losing no detail.
 - The client can potentially catch back up if:
 - Rate of append (i.e., the rate of arrival of information relevant to the stream) reduces due to conditions in the monitored environment changing.
 - The client gains additional resources that allow it to deal with greater than the current rate of append.
- Client read delay is greater than the compaction delay but less than the tombstone retention time.
 - Client is behind on absolute state and is losing fidelity as some changes are being compacted out of the log (if the client is less interested in short lived things than long lived things this may not be a significant problem)
 - A rapid intermittency may become completely invisible (e.g., an active – clear alarm pair)
 - All changes that happen at a rate slower than read delay will be visible.
 - The client can potentially catch back up if:
 - The delay was due to a long comms down issue that has now recovered, and the client capacity can readily deal with the current append rate.
 - Rate of append reduces due to conditions in the monitored environment changing.
 - The client gains additional resources that allow it to deal with greater than the current rate of append.
- Client read delay is greater than the tombstone retention time.
 - Client has now potentially lost the eventual consistency and must realign by streaming from the “oldest” record (offset zero)

3.18 Stream Monitor

TAPI 2.1.3 also offers a rudimentary capability for monitoring the streams. This allows an external stream administration client that has the appropriate capability and authorization to monitor which clients are connected and how their stream is performing.

Where this capability is supported for each client, streaming connection is monitored for the id of the last record written to and read from the log. This allows an administrator to get a view of how delayed a client is.

In TAPI 2.1.3 this is for PoC analysis. It is expected that the feature will advance significantly in future releases as a result of experience gained from PoC activity.

³¹ This is the duration that “tombstone” records are retained for. This prevents the log size being unbounded.

³² This is the duration that all records are guaranteed to be persisted for (after they are logged) before compaction will consider removal based upon more recent logging for the same entity.

3.19 Solution structure – Architecture Options

Two options are explored, one provides full compaction support, the other uses a more traditional structure to feed a stream and provides a restricted emulation of compaction. Either can be used to support the current TAPI streaming solution sufficiently.

There may be other approaches that provide a suitable capability, i.e., reporting current state and change via whole entity reporting through the same single stream, so as to achieve eventual consistency with current state, cost-optimized for potential loss of fidelity.

The key consideration is the external behavior and not the specific mechanism used to support it.

3.19.1 Full compacted log

The figure below shows a stylized view of a controller controlling a network (the controlled system) and providing a stream to a client.

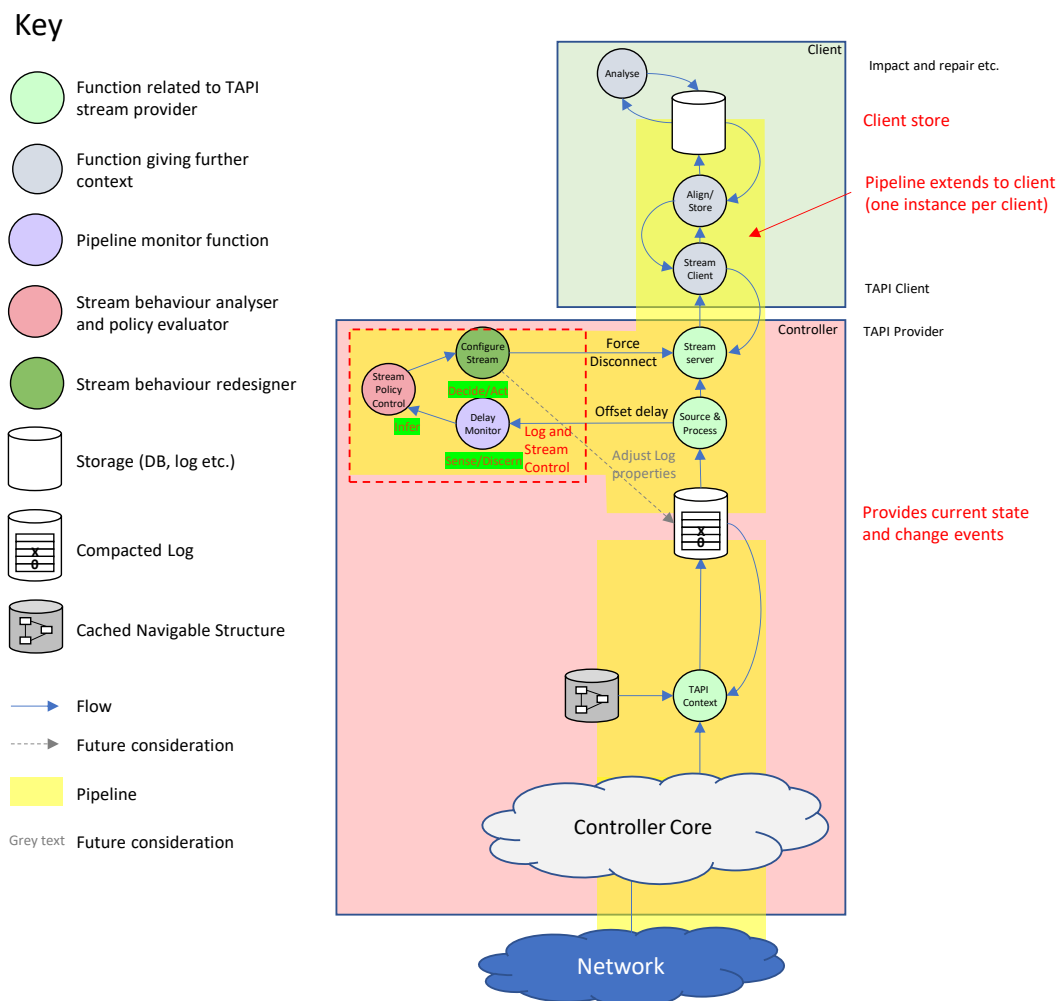


Figure 7 Stylized view of example controller offering full compaction.

The key features highlighted in the diagram are:

- Compacted log providing current state, recent Tombstones and recent changes.
 - The compacted log records whole entity instances on change
 - See [Appendix 1 Compaction using Kafka \(as part of an example implementation\)](#)
- Pipeline providing guaranteed delivery (at least once) whilst connection in place along with provider-initiated connection force drop control when realignment is necessary.
 - To align the client does not need to do anything other than connect to the appropriate stream and receive from the stream.
 - It is assumed that the client will extend the integrity of the stream to include the process of storage (as shown) so as to enable the client to maintain alignment with current state (eventual consistency) and to build and maintain history.
- A stream monitor (depicted as a control loop on the pipeline as an aspect of control of control) that ensures achievement of eventual consistency.
 - Delay Monitor senses the age of the records being streamed (delay)
 - Stream Policy Control infers whether the current delay is acceptable. This depends upon what state the stream is in (e.g., start-up) etc. It provides input to Configure Stream.
 - Configure Stream decides what action to take and then takes that action. This may include forcing a disconnect to cause a client-reconnect and subsequent realignment and/or adjustment of Log properties.

Note that the assumption is that other pipelines are in place (not shown here) throughout the overall flow from controlled device to the client to ensure no relevant loss of information from the controlled device and to ensure that the solution is always in a state of eventual consistency with current network state.

3.19.2 Emulated compaction

This approach uses the same pipeline mechanism but feeds the pipeline from a composite store. This does not offer the full compacted log capability. The behavior is as if the tombstone retention and the compaction delay are set to the same value (simplistically, the end of the log, but strictly any point in the log).

On client connection:

- The provider would start the “time-truncated” log to collect all changes that occur from the point of connection³³
 - These changes would be recorded as whole entity snapshots.
- Start to stream current state.
 - Current state should include time of occurrence of last change.
 - To achieve this the provider may need to page through a large store of data such that the state is skewed over some, potentially long, period of time.

Current state may change during streaming of current state such that a change statement (whole entity snapshot) is appended to the truncated log. Hence it is quite likely that the change log will have changes in that have also been provided as part of the current state.

Once the provider has sent current state, ensuring that all states that have not changed since connection of the client have been streamed to the client, it would then begin to stream from the truncated log. The truncated log may include some states that have already been streamed. As the client is necessarily designed to be able to deal with repeated statements this will not be an issue.

³³ It should be feasible for a single truncated log to be used for multiple clients, however, the current state will need to be dealt with on a per client basis.

If the connection to the client is dropped, the client will reconnect providing the token of the last record it fully processed. The provider can code into the token any information that helps it determine what to stream next.

Clearly, the client must be able to retrieve current state in a shorter period than the log truncation time so that changes are not lost. If the log wraps during streaming of current state, the provider will have to restart the alignment (by dropping the connection and by ignoring the client token).

It would not be unreasonable for the provider to shorten the truncated log once alignment has been achieved.

The provider is expected to add a Tombstone record for every delete record. The emulated compacted log could avoid using the specific tombstone record as there is no log size benefit of retaining only a compressed record.

It should be possible for the provider to use one single truncated log for all clients, however on initial connection of a client it will be necessary for the provider to construct a specific current state snapshot to feed the stream to that client.

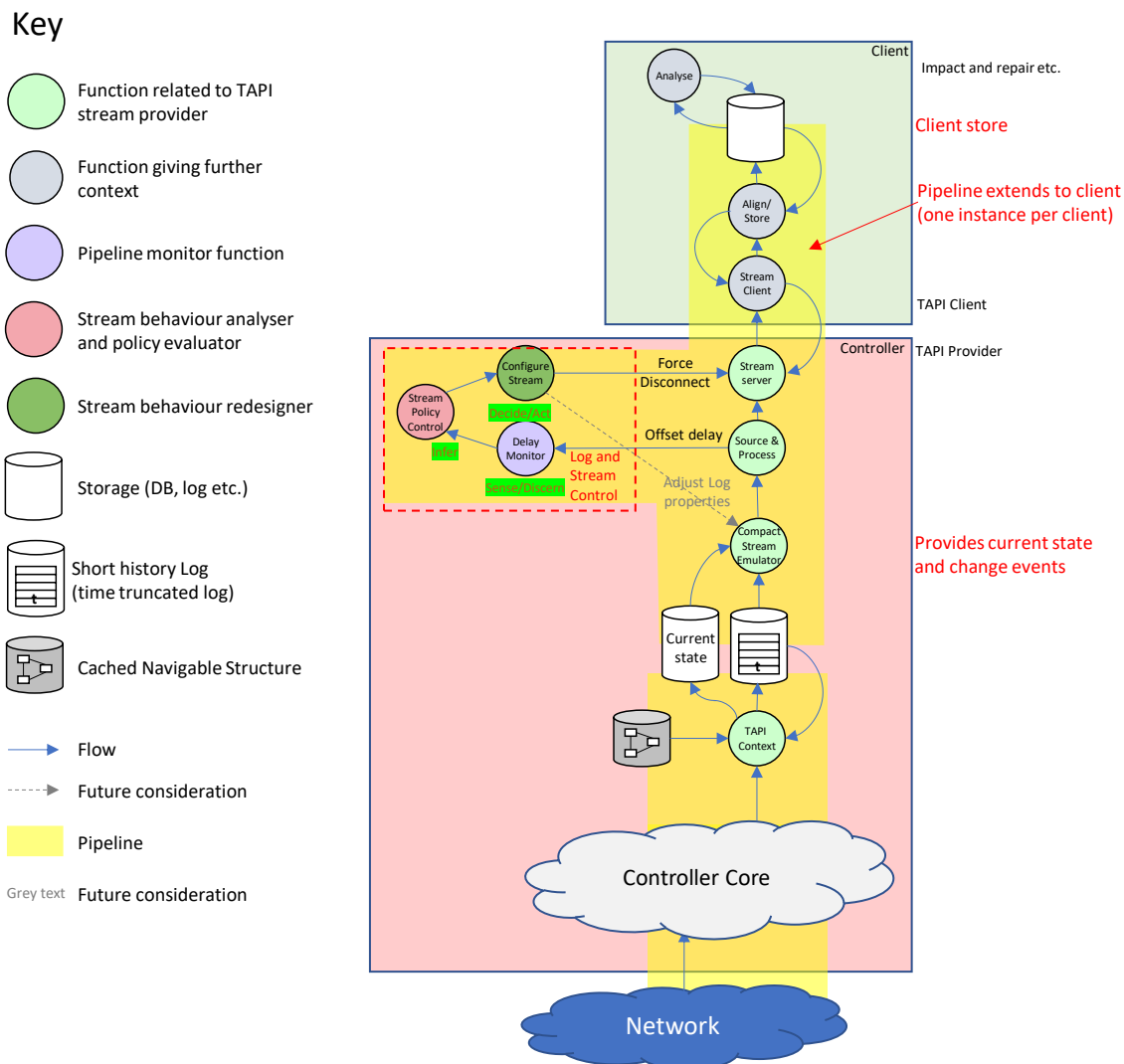


Figure 8 Stylized view of example controller offering emulated compaction

The provider behavior is feeding the stream is very similar to a traditional behavior (send current in response to get then notify of change, other than the log is of whole entities).

In this realization the provider side does not carry out active compaction and hence does not provide the elegant, degraded performance of the full compacted log approach when the client is under pressure and significantly delayed.

3.19.3 Comparing the full compacted log and the emulated compacted log

From the client perspective the emulated compacted log appears to be a compacted log with the compaction delay equal to the tombstone retention.

The emulated compacted log is also more likely to have a tombstone retention determined by record count as opposed to time.

4 Using the compacted log approach for alarm reporting

This section provides both informative and normative statements on the alarm solution.

4.1 Specific alarm characteristics - raising/clearing an alarm

A controlled device raises an alarm once specific criteria have been met and clears the alarm once other specific criteria have been met. The criteria could involve, for example:

- Counting traffic related events where the count is within some window (often sliding and sometimes complex)
 - The alarm will be raised when the count exceeds some threshold and will be cleared when it drops below some threshold.
 - The two thresholds provide hysteresis that brings some stability to the alarm.
 - The windows are often very short and can cause extreme intermittency under particular network scenarios.
- Measuring an analogue property with several options
 - The alarm may be raised when the measurement exceeds some threshold and cleared when it drops below another threshold with hysteresis.
 - The alarm may be raised when the measurement drops below some threshold and cleared when it raises above another threshold with hysteresis.

The counts can be of the occurrence of other threshold crossing and hence the alarm definition at origin may be very complex.

4.2 Key Features of an alarm solution (example usage)

The following sections work through the key features of an alarm solution as an example of usage of streaming.

The alarm streaming solution has a particular delete/tombstone behavior to provide the best performance. This is highlighted in 4.7 Alarm tombstone behavior on page 39.

4.3 Log strategy

This section identifies the key characteristics of the alarm log and stream and summarizes the implications.

1. The TAPI alarm stream shall be fed from a compacted log.
2. There should be a dedicated connection through which only alarm records are propagated.
3. The log compaction delay shall be set to allow for normal operation with a client to enable the system to deal with temporary communications failure with no loss of fidelity.

- When compaction is applied fidelity will be lost, although only rapidly changing and fleeting alarms will be lost.
 - In any solution, the client may be a little behind or may suffer a short communication disruption without significant impact on operational quality.
 - The proposed compaction delay setting is 10 minutes³⁴ for a system with reasonably well engineered platform capacity and communications.
 - Clearly having an alarm system that is greater than 10 minutes behind the provider will degrade location performance, however experience may show that this time is too short.
 - The solution may allow for this property to be adjustable in the running system, through a mechanism suitable for expert access.
 - Dynamic compaction delay might be beneficial in cases where local control can be applied, and an unusually long comms down is being experienced.
 - Under these circumstances the compaction delay could be increased such that the clients lose no fidelity (although they are clearly behind whilst the comms is down)
 - Note that:
 - ideally, during the normal operation, the client would be at most a few minutes behind the current append time.
 - the volume of information within the compaction delay time depends upon the rate of change of things in the monitored environment.
 - in a sophisticated solution it will be possible to allocate more resources to the client application, when it is under pressure, to enable it to process faster.
4. The log tombstone retention shall be set to allow for reasonable communication failure/recovery where there may be significant failures and to allow for client reboot.
 - The proposed tombstone retention setting is 4 hours for a system with reasonably well engineered platform capacity and communications.
 - This solution may allow for this property to be adjustable in the running system, through a mechanism suitable for expert access.
 - Tombstone retention will always be greater than or equal to the compaction delay.
 5. The normal record (non-tombstone) retention shall be infinite.
 - This property is not adjustable as it is fundamental to the correct operation of the mechanism
 6. If compaction operates on a segment by segment³⁵ basis such that the segment with the most recent alarms is never compacted (see section 6.1.3 Log segments on page 70), then the segment size shall be such as to not hold significantly more records than would occur during the compaction delay time when operating under normal conditions (ideally a segment would hold significantly less).

Also see section 3.16 System engineering on page 29.

4.4 Alarm behavior

The following highlights key design decisions (in the context of justifying/explanatory information) and should be read in conjunction with the background provided in section 6.4 Appendix – Detectors, detected conditions and alarms on page 74:

³⁴ Assuming that an acceptable system will be engineered to not be any more than 10 minutes behind under bad-day conditions. Note that this parameter can be tuned to suit system engineering and also desire to achieve full fidelity. A longer compaction delay will cause there to be more recent history in the initial alignment and hence may slow initial alignment. A similar behavior occurs for a traditional notification queue. It is possible to dynamically tune the stream to reduce this impact.

³⁵ The log (on disc) saves records in a live area, a segment, that has a defined size. Once the size is reached a new segment is started for saving new records and the old segment is moved to a historic state. This old segment is available for compaction.

1. An alarm detector shall have an ACTIVE and a CLEAR state
 - Explanatory Information:
 - An active is considered as more important than the clear (hence the states are asymmetric in importance)
 - Most detectors in the network will be clear under normal circumstances (hence the normal state can be considered as clear and the off-normal state as active)
 - The alarms shall be reported as ACTIVE or CLEAR.
 - An ACTIVE will be followed at some point by a CLEAR (the basic state machine is simple with only Clear → Active and Active → Clear transitions)
 - An alarm CLEAR shall be followed immediately by a Tombstone.
 - A Tombstone alone shall be considered as equivalent to a clear
 - The tombstone causes the alarm to be removed from the log. This then has a similar characteristic to a traditional alarm reporting mechanism.
2. If an entity upon which the detector is lifecycle dependent is deleted and just prior to the deletion the alarm was active, then the alarm (and hence its detector) shall be tombstoned.
 - If the alarm was not active, then there shall be no Tombstone as there is nothing in the log to remove.
3. The output of a particular detector may be processed and stabilized such that it gains a state, INTERMITTENT, indicating that the detector is intermittently rapidly cycling through active and clear states.
 - The alarms will be reported as ACTIVE, INTERMMITTENT or CLEAR
 - All transitions are legal (i.e., Clear → Active, Clear → Intermittent, Active → Clear, Active → Intermittent, Intermittent → Active, Intermittent → Clear)
 - The trigger for moving to/from Intermittent state will be defined by policy. Ideally the policy is configurable.

Note: It would be reasonable to also propagate other processed network property types through the same stream as alarms if the network property has similar characteristics to an alarm. For example, operational state is asymmetric in importance with a normal state and off-normal state where the normal state could be considered as equivalent to a clear.

4.5 Condition detector and alarm structure

The condition-detector is as below (descriptions omitted to save space).

```

grouping condition-detector {
  leaf condition-native-name {
    type string;
    config false;
  }
  leaf measured-entity-uuid {
    type tapi-common:uuid;
    config false;
  }
  leaf measured-entity-native-id {
    type string;
    config false;
  }
  leaf condition-normalized-name {
    type string;
    config false;
  }
  leaf measured-entity-class {
    type object-class-identifier;
    config false;
  }
  leaf detector-uuid {
    type tapi-common:uuid;
    config false;
  }
  leaf detector-native-id {
    type string;
    config false;
  }
  leaf condition-detector-type {
    type condition-detector-type;
    config false;
  }
  leaf-list measured-entity-local-id {
    type string;
    config false;
  }
}

```

Figure 9 Yang: condition-detector (descriptions omitted)

For this structure a solution shall provide the following support:

- **Mandatory:**
 - condition-native-name³⁶: The name used for the condition at the source.
 - measured-entity-native-id³⁶: The identifier (invariant over the life) of the instance of the measured entity at the source.
 - detector-native-id: The identifier (invariant over the life) of the instance of the detector at the source (e.g., a network device).
 - condition-detector-type: Identifies the type of detector. This drives the conditional augmentation.
- **Conditional Mandatory:**
 - condition-normalized-name: Commonly used or standardized condition name.
 - Mandatory if there is a normalized name available for the condition
 - measured-entity-class: The TAPI class of the measured entity.
 - Mandatory if the condition relates to a TAPI entity³⁷

³⁶ It is vital that the alarms stated on TAPI reflect what would be seen at the device regardless of the mappings etc. such that the operator can reference the vendor definitions etc.

³⁷ Some devices report alarms related to entities external to the device. In these cases, there is no TAPI entity to report against.

- measured-entity-uuid: The UUID of the TAPI entity that represents the entity measured at source.
 - Mandatory if the alarm relates to a TAPI entity
- measured-entity-local-id: The local-id of the TAPI entity that represents the entity measured at source.
 - Mandatory Where the measured entity is a local class, and hence does not have a UUID, the local ID is provided in conjunction with the parents UUID (in the measured-entity-uuid property).
- detector-uuid³⁸: The uuid of the TAPI entity that represents the detector.
 - Mandatory where the detector has a UUID.

The condition-detector can be augmented with properties related to alarms as follows:

```

grouping alarm-condition-detector-detail {
  leaf alarm-detector-state {
    type alarm-detector-state;
    config false;
  }
  container legacy-properties {
    config false;
    uses legacy-properties;
  }
}

...

grouping legacy-properties {
  leaf perceived-severity {
    type perceived-severity;
  }
  leaf service-affect {
    type service-affect;
  }
  leaf is-acknowledged {
    type boolean;
  }
  leaf-list additional-alarm-info {
    type string;
  }
}

```

Figure 10 Yang: alarm-detector and legacy-properties (descriptions omitted)

For this structure, a solution shall provide the following support:

- Mandatory:
 - alarm-detector-state: indicates whether the alarm is ACTIVE, INTERMITTENT or CLEAR. This is the essential state of the alarm.
- Optional (see following text):
 - perceived-severity
 - service-affect
 - is-acknowledged

Many solutions and operational practices continue to use the historic schemes (see section 6.4.8 Traditional alarm reporting and legacy-properties on page 75). On that basis, the schemes are supported but relegated to

³⁸ The detector may have a distinct UUID or may simply have a UUID constructed from that of the entity for which detection is being performed (e.g., a CEP) along with the detector name.

optional. At this point in the evolution of control solutions legacy-properties are probably mandatory, however, it is anticipated that as control solutions advance the legacy-properties will become irrelevant.

The legacy-properties are:

- **perceived-severity:** A network device will provide an indication of importance for each alarm. This property indicates the importance. In some cases, the severity may change through the life of an active alarm.
- **service-affect:** Some network devices will indicate, from its very narrow viewpoint, whether service has been affected.
- **is-acknowledged:** Network devices offer a capability to acknowledge alarms (to stop the bells ringing). Often an EMS will offer a similar capability. This property reflects the current acknowledge state.
- **additional-alarm-info:** Often, alarms raised by network devices have additional information. This property can be used to convey this.

Although originating from network device control, the above properties may be supported by other device types.

4.6 Alarm Identifier and location

This section further clarifies the rationale for the choices of mandatory and optional identifiers and provides further requirements.

- The identifier of the alarm is essentially the identifier of the detector (**detector-native-id**). This shall be based upon native device values to ensure consistency over time and across resilient systems etc.
 - The detector is at a functional location in the solution and detects a particular condition at that location. It shall be identified by these two key properties. i.e., functional location and condition.
 - The detector is long lived and may emit many active and clear events through its life.
- The alarm will normally be reported via TAPI against a valid TAPI entity and hence the overall location of the detector will include the identifier of the TAPI entity.
 - The report shall also include information in the location from the device in device terminology to enable the relating of information acquired directly at the device with information acquired at a higher system.
 - The TAPI model allows for alarms without a full TAPI resource id, although this should be a rare case.
- Where the detector relates to a thing that is not fully modeled in TAPI, e.g., a power supply, then:
 - The alarm shall be reported against a containing TAPI entity.
 - The identifier of that detector shall include a meaningful index that include interpretable sub-structuring to describe the position of the detector (again based upon device values)

4.7 Alarm tombstone behavior

As noted earlier, the alarm clear shall be followed immediately by a Tombstone record. As also noted, the deletion of an alarm detector, where the alarm was active prior to the deletion, shall cause at least the logging of a Tombstone record. Allowing for compaction delay:

1. The Tombstone/clear shall cause the compaction process to remove the corresponding active alarm.
2. The Tombstone shall also cause the compaction process to remove the clear (that immediately precedes it)
3. The Tombstone shall eventually be removed as a result of tombstone retention duration being reached.

4.8 Time

Whilst the timestamp for the alarm is not a provider responsibility (other than for provider generated alarms, e.g., disc full), there is an expectation of correct behavior.

A versatile approximate-data-and-time structure has been used for the event-time-stamp to allow representation of inaccuracies.

The time stamp for an alarm shall be as follows:

- The timestamp from the ultimate source, representing the time of occurrence of the event that triggered the raising or clearing of the alarm, shall be preserved through the entire passage of the alarm information through the source device, and any chain of controllers, and presented in the appropriate field in the TAPI log-record-body.
 - Where the source does not provide a time stamp, a timestamp shall be added by a provider as the alarm progresses through the solution. This timestamp shall be marked with a spread value BEFORE
- In general, it is extremely important to accurately capture the leading edge of a problem. Hence, for each detector, the time of first occurrence of an active alarm after a "long" period of clear is the most significant time.
 - The time of clearing can be less precise.
- The log record also has a log-append-time-stamp.
- Ideally, the time of occurrence of an alarm should be the time of the entry into the confirmed assessment as to whether an alarm has occurred (and not the time of exit from that assessment).
 - Likewise, the time of clearing of the alarm should be the time of entry into the confirmed assessment as to whether an alarm has cleared.

4.9 Detected Condition normalization

The alarm detected condition shall be presented:

- Minimally: In native form, i.e., as provided by the Device
- Additionally: In normalized form, i.e., complying with a list of standard alarm types

An alternative is to provide translation metadata to enable normalization from the native form at the client. This metadata can be provided separately from the alarm stream and related to each detector with a relevant mapping. There is not standard expression for translation metadata.

4.10 Meaningful detection (device or any other source)

Under certain circumstances a detected condition may:

- Become meaningless, e.g., when a remote function is disabled, in which case the alarm shall be tombstoned.
 - This may require function disable action to be taken on the local system.
- Have inverted meaning, e.g., when signal should not be present on a port, in which case, if there is a "signal not present" alarm active, then it should be tombstoned and if there is a signal present a "signal present" alarm should be raised.
 - This may result from some local action on the entities supporting signal flow.
 - Essentially, the "signal not present" condition detection becomes meaningless, and a "signal present" condition detection becomes meaningful.

5 Use Cases

5.1 Use Case General Considerations

5.1.1 TAPI Context

Initial TAPI deployments using TAPI 2.1.3 are applied to solutions where a provider is feeding a client from a single context that is taking a view of the entire controlled network for all layers.

5.1.2 Underlying behavior

It is assumed that at start up the provider will form the necessary full context and will initialize various streams. It is also assumed that the provider will recover from any internal problems to ensure that the streams allow the client to achieve eventual consistency.

There are a number of critical behaviors assumed from the underlying system (essentially use cases for other components):

- TAPI is fed from a reliable source that has necessary notifications and access to current state etc. (e.g., has alarm notifications).
 - I.e., the underlying system through the provider to the network device is reliable (appropriate pipelines, etc.) so that the provider cannot lose eventual consistency with the Devices.
 - The solution may use compacted stream in which case the compaction delay and tombstone retention are compatible with TAPI needs.
- The notifications are well behaved both at the network device level and within the provider, e.g., for alarms such that:
 - An alarm will have a defined active and a defined clear.
 - Only legal transitions (clear to active and active to clear etc.) are represented.
- If the resource is deconfigured³⁹/deleted a Tombstone will be logged for each dependent resource (e.g., alarm detector) related to the resource that was indicating active just prior to the deconfiguration/deletion.
- If a circuit pack is configured the states of any dependent resources will be reported appropriately, e.g., if an alarm detector indicates active an active alarm record will be logged.
- If a circuit pack is deconfigured any dependent resource will be tombstoned, e.g., any dependent alarm detectors that are active will have Tombstones logged.

5.1.3 Model conformance

Where a compacted log stream is used for gaining and maintaining alignment (see ST-03), the entities should comply with TAPI Yang and with the “Relevant parameters” definitions in the specific use case set out in [LF TR-547] as explained in the following table.

Note that some entity types:

- vary in parameters depending upon the layer-protocol (as described in [LF TR-547] use cases).
- have additional NVP properties defined in [LF TR-547].

³⁹ Most, but not all deconfiguration actions will result in removal of entities, hence some deconfiguration actions will not cause tombstones.

Entity Type	Required Parameters stated in [LF TR-547] Use Case	Comments
context	Use Case 0a	
service-interface-point	Use Case 0a	Depends upon layer protocol choice.
topology	Use Case 0b	
node	Use Case 0b	
node-edge-point	Use Case 0b, 0d	Depends upon layer protocol choice.
node-rule-group	Use Case 0b	
rule	Use Case 0b	
link	Use Case 0b	
connectivity-service	Use Case 1.0, 1a, 3a, 3b, 3c, 3d, 3e, 3f, 5b, 5c, 6a, 6b, 7a, 7b, 8, 9	
connectivity-service-end-point	Use Case 1.0, 1a, 1c, 1g, 2a, 2b, 2c, 2d, 5b	Depends upon layer protocol choice.
connection	Use Case 1.0, 5b	Including switch control and switch
connection-end-point	Use Case 1.0, 1c	Depends upon layer protocol choice.
route	Use Case 1.0	
equipment	Use Case 4b	
device	Use Case 4b	
path-computation-context	Use Case 12a	
path-computation-service	Use Case 12a	
path-service-end-point	Use Case 12a	
topology-constraint	Use Case 12a	
routing-constraint	Use Case 12a	
objective-function	Use Case 12a	
optimization-constraint	Use Case 12a	
oam-service	Use Case 17a	
oam-service-point	Use Case 17a	
meg	Use Case 17a	
mep	Use Case 17a	
mip	Use Case 17a	
pm-current-data	Use Case 17a	
pm-history-data	Use Case 17a	
pm-threshold-data	Use Case 17a	

5.1.4 Use Case Overview

The use cases are divided into 6 groups. Use Case group:

- ST-0.n deal with streaming infrastructure
- ST-1.n deal with building and operation a stream on a provider
- ST-2.n deal with strategies for the client to maintain alignment with the provider
- ST-3.n detail the approach for the client to maintain alignment with resources and with alarms
- ST-4.n deal with whole context considerations
- ST-5.n deal with streaming through the lifecycle of a service

5.2 Streaming infrastructure use cases

The following use cases are described briefly here and then illustrated in the sequence diagram (see Figure 11 and Figure 12). The use cases assume that Websockets over TCP is the chosen connection/protocol method.

The following uses cases are initiated roughly in the order set out below. The interdependence between the use cases is illustrated by the sequence diagram. The use cases in this section are normative.

5.2.1 Use Case ST-0.1: Get Auth Token

Number	ST-0.1
Name	Get Auth Token
Process/Area	Authorization and Authentication
Brief description	This use case describes how the client acquires the Auth Token.
Preconditions	The client knows the provider address and where to get the Auth Token
Type	Acquiring information
Description and workflow	The client acquires the Auth Token, for example, using method described in 5.10.2. See sequence diagram “Prepare” phase (Figure 11 and Figure 12).

5.2.2 Use Case ST-0.2: Discover supported and available streams, then select available streams

This use case deals with retrieval of data conformant with 3.7.

Number	ST-0.2
Name	Discover supported and available streams, then select available streams
Process/Area	Streaming infrastructure
Brief description	This use case describes how the client acquires information on supported and available streams.
Preconditions	UC ST-0.1 has run successfully
Type	Acquiring information
Description and workflow	The client gets all supported-stream-type structures from the context. The client examines record-content for streams that support entities or combinations of entities of interest. For each stream type that supports the appropriate entity/entity combination, the client examines: 1. log-storage-strategy: to identify a stream that has the right characteristics (e.g., COMPACTED) 2. log-record-strategy: to identify a stream that has the right record characteristics (e.g., WHOLE_ENTITY_ON_CHANGE) 3. Other log parameters to tune its operation to suit timer settings etc. The client examines available-streams to find running streams that reference the stream-types identified in supported-stream-type-ref. For each available-stream that is of a relevant stream-type, the client examines: 1. stream-state: to determine if the stream is operating. 2. connection-protocol: to identify a stream that offers a compatible protocol for connection (e.g., Websockets) 3. connection-address: to determine where to connect

5.2.3 Use Case ST-0.3: Connect to Stream and align - new client

This use case deals with use of the stream which is explained further in 3.9 with content as in 3.10 and behavior as described in 3.11, 3.12, 3.13, 3.14, 3.16 and 3.17.

Number	ST-0.3
Name	Connect to Stream and align - new client
Process/Area	Streaming Infrastructure
Brief description	This use case describes the connection of a client to a stream and the client acquiring stream records.
Preconditions	UC ST-0.1 has run successfully. The client has gained knowledge of the available streams via some mechanism (such as UC ST-0.2)
Type	Gaining and maintaining alignment
Description and workflow	The client uses the results from UC ST-0.1: 1. Use the provided endpoint address and method to connect. • The client will connect with the null token⁴⁰ to cause the provider to stream from the oldest record in the stream (offset zero) 2. On connection both the client and provider stream processes are started, and the necessary communication is setup between client and provider. • As a result, the pipeline is started. • The provider reads the appropriate log from offset zero filling the pipeline and responding to ongoing demand. 3. The client demands from the provider and buffers as appropriate. 4. The client stores in a repository (e.g., Log, database etc.) 5. The client maintains a record of last commit (see also ST-2.1) Through the above activities the client works through the stream from initial record towards the most recent record 6. Passing the tombstone retention "point" • It must take less than the tombstone retention period to reach the tombstone retention "point" in the log. • If it takes longer than the tombstone retention the connection will be dropped as Tombstones for records already read may have been missed. 7. Passing the compaction "point" • From this point onward the client will be getting a full view of changes. 8. Getting close to the head of the stream • The client is well aligned with little lag. Assuming that the client is engineered to match the provider stream rate, the client should achieve a steady state and continue to be reading records close to the head of the stream, i.e., close to the most recent record logged by the provider. See sequence diagram “Connect” and “Streaming” phases where the stream starts from “offset 0” (Figure 11 and Figure 12). As changes occur in the network etc. records of the changes are appended to the appropriate logs and then streamed to the client.

⁴⁰ The token referred to here is the token from the log-record-header (see 3.10 Record content on page 23)

5.2.4 Use Case ST-0.4: Client maintains idle connection

Number	ST-0.4
Name	Client maintains idle connection
Process/Area	Streaming Infrastructure
Brief description	This use case describes how the client will maintain an idle connection
Preconditions	UC ST-0.3 has run successfully
Type	Maintaining Connection
Description and workflow	1. The client uses protocol specific mechanisms to ensure that an idle connection is maintained open. • E.g., the client periodically sends a uni-directional "Pong" frame on the connection (https://tools.ietf.org/html/rfc6455#section-5.5.3) in order to keep an idle WebSocket connection open. • No response expected from the server. • The server should expect a pong frame with appropriate timing. As shown in Figure 11.

5.2.5 Use Case ST-0.5: Provider delivers event storm (or slow client) – bad day

The client takes advantage of log behavior as described in 3.17.

Number	ST-0.5
Name	Provider delivers event storm (or slow client) – bad day scenario
Process/Area	Streaming Infrastructure
Brief description	The provider(s) record events at a higher rate than the client can handle
Preconditions	UC ST-0.3 has run successfully
Type	Gaining and maintaining alignment
Description and workflow	Note that the use case is written in general. A relatively common network example related to alarms can be used. This example includes major intermittent failure in the network, for example, several micro-bends with active mechanical vibration interference, overloaded client with reduce compute power available. 1. The pipeline continues but the client is absorbing events at a rate slower than the production and hence is slipping back down the log. 2. Eventually the client will slip back beyond compaction point. 3. If the problem resulted from excessive intermittent network activity the client will then benefit from compaction as much of the intermittent noise will be eliminated by compaction • The client will lose fidelity and will be sub-Nyquist sampling⁴¹ so may completely lose some repeated fleeting events but regardless of the scheme used, the client would not be able to maintain full alignment with history. 4. The client may hover in the compaction zone until its performance improves (via some mechanism for compute power enhancement) or the network stabilizes. • Note that the intention in future is to support sophisticated backpressure interaction controlling intelligent filtering in the devices and network⁴². This would be coordinated by the provider as client load problems are detected. See sequence diagram “Streaming” phase (Figure 11 and Figure 12).

5.2.6 Use Case ST-0.6: Provider delivers extreme event storm (or very slow client) – very bad day

The client takes advantage of log behavior as described in 3.17.

⁴¹ See https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon_sampling_theorem

⁴² Note that this will be client induced server behavior. This assumes a single client scenario for TAPI... there is a single alarm system etc. (perhaps resilient). It can work for multiple clients so long as there is a reasonable balance of engineering and some priority scheme etc.

Number	ST-0.6
Name	Provider delivers extreme event storm (or very slow client) – very bad day scenario
Process/Area	Streaming Infrastructure
Brief description	The provider(s) record events at an extremely high rate, much higher than the client can handle
Preconditions	UC ST-0.3 has run successfully
Type	Gaining and maintaining alignment
Description and workflow	Note that the use case is written in general. A relatively common network example related to alarms can be used. This example includes extreme intermittent failure in the network, for example, timing faults and micro-bends with active mechanical vibration interference, and a massively overloaded client with reduce compute power available. Streaming continues: 1. The pipeline continues as before, but the client rapidly slips back past the compaction point toward the tombstone retention point. • If the client passes tombstone retention, then there is a possibility of loss of eventual consistency as deletes will be lost. Failure occurs: 2. On passing the tombstone retention point the provider forces a disconnect by dropping the connection • The client and the provider kill the pipeline. Realign: See UC ST-0.9 See sequence diagram “Streaming” and “Drop Connection” phases (Figure 11 and Figure 12).

5.2.7 Use Case ST-0.7: Short loss of communications

The client takes advantage of log behavior as described in 3.17.

Number	ST-0.7
Name	Short loss of communications
Process/Area	Streaming Infrastructure
Brief description	The communications between the client and provider fails briefly (less than tombstone retention time) then recovers
Preconditions	UC ST-0.3 has run successfully
Type	Gaining and maintaining alignment
Description and workflow	1. The client is consuming the stream with some delay. 2. On loss of comms the client and the provider kill the pipeline. 3. The log continues to progress on the provider side. 4. The client attempts to reconnect with the token of the most recently processed record. • This will be successful assuming the comms has recovered and the time that the client has been disconnected from the provider is no longer than the tombstone retention time. 5. The stream is filled by the provider from record with the next valid token after the token provided. • In a short comms loss case, this token will be for the next record logged by the provider. • The record with the provided token will still exist in the log. • In a longer comms loss, compaction may have taken place such that the next valid token is for a record that was not logged adjacently to the record with the token provided. • The record with the provided token may no longer exist in the log as a result of compaction. 6. Assuming that the client was near the most recent record of the log there should be no loss of fidelity (and clearly eventual consistency will be maintained) • If the client was in the compaction zone, then there will be some loss of fidelity (see UC ST-0.6) • If the client was close to tombstone retention, then the short comms loss may have the behavior of a long comms loss (see UC ST- 0.8) See sequence diagram “Streaming”, “Loss”, “Connect” (with specific token) and “Streaming” phases (Figure 11 and Figure 12).

5.2.8 Use Case ST-0.8: Long loss of communications requiring realignment

The client takes advantage of log behavior as described in 3.17.

Number	ST-0.8
Name	Long loss of communications
Process/Area	Streaming Infrastructure
Brief description	The communications between the client and provider fails then recovers
Preconditions	UC ST-0.3 has run successfully
Type	Gaining and maintaining alignment
Description and workflow	1. The client is consuming the stream with some delay. 2. On loss of comms the client and the provider kill the pipeline. 3. The log continues to progress on the provider side. 4. The client/comms is down for longer than the tombstone retention. 5. The client attempts to reconnect with the token of the record that it previously successfully processed. 6. The provider recognizes that this is outside tombstone retention and streams from the oldest record etc. • The client could take action to revert to the oldest record by not providing the token, however, it will certainly take longer to align than relying on the client knowledge regarding whether alignment is required or not. 7. See UC ST-0.9 See sequence diagram “Streaming”, “Loss”, “Connect” (with specific token but forced to offset = 0) and “Streaming” phases (Figure 11 and Figure 12).

5.2.9 Use Case ST-0.9: Client requires realignment

Number	ST-0.9
Name	Client requires realignment
Process/Area	Streaming Infrastructure
Brief description	For some reason, the provider determines that the client needs to be aligned/realigned or the client chooses to realign
Preconditions	UC ST-0.3 has run successfully
Type	Gaining and maintaining alignment
Description and workflow	1. The client reconnects with the previous token. • Or connects with no token. 2. On connection both the client and provider stream processes are started, and the necessary connection is made between client and provider. • As a result, the pipeline is started. 3. The provider recognizes the token is outside tombstone retention and restarts the stream from the oldest record. 4. The provider informs the client, via the stream, that it is back at oldest record (offset zero) 5. The client consumes the stream to regain alignment. • If the provider is logging records at a significantly higher rate than the client can handle, the client will stay beyond the tombstone retention and will get forced to realign. • The client may utilize some efficient realignment strategies (see ST-2.2) • The client is expected to use some form of “garbage collection” mechanism to ensure that instances that are stored by the client but that are no longer present in the provider stream are eliminated. Assuming that the reason for the stream restart have gone, then the client will regain alignment (eventual consistency) and will return to the state achieved in UC ST-0.3 See sequence diagram “Connect” (with specific token but forced to offset = 0) and “Streaming” phases (Figure 11 and Figure 12).

5.3 Building and operating a stream on a provider

The following use cases describe how the provider should populate the stream. The specific details of storage and internal mechanism are simply examples, however, the data in the log is essentially normative.

The use cases in this section are normative, except use case ST-1.3.

5.3.1 Use Case ST-1.1: Provider initializes and operates a stream

Number	ST-1.1
Name	Provider initializes and operates a stream
Process/Area	Streaming operation
Brief description	The provider initializes the streams from internal sources of current state and change
Preconditions	The provider is running, the streaming infrastructure is running and there is a defined stream to populate. Note: As the activities associated with achieving the preconditions will vary significantly from solution to solution and are internal detail, no use cases have been provided for this area.
Type	Preparing and maintaining a stream
Description and workflow	- The stream type is added to the supported-stream-type structure if not already listed. - The data shall comply with the structure set out in section 3.7.1 - The provider creates a stream for a specific information source. - The stream is added to the available-stream structure with a stream-state of “ALIGNING”. - The data shall comply with the structure set out in section 3.7.2 - A client may connect to the stream from this moment on - In this state no records will be streamed to a connected client - The provider populates the stream from internal stores of current state and change. - The data shall comply with the structure set out in section 3.10 [example] If the provider uses compacted-log-based streams internally, this may simply be connecting to the appropriate internal streams from the oldest record and performing necessary transformations to form the TAPI stream content. [example] If the provider uses a current state repository with a truncated stream of changes, then it will populate the stream with appropriately transformed current state followed by recent changes. - It is possible that the current state repository does not record event time. If this is the case, the event-time-stamp should take advantage of the approx-date-and-time structure. The event-time-stamp should be the same as the log-append-time-stamp with the spread set to “BEFORE”. - For current state and for change notifications the provider appends a record of the whole entity instance using the appropriate structure for the class with: - record-type set to CREATE_UPDATE. - record-content set to the appropriate object-class-identifier. - entity-key set to a value appropriate for identifying instances of the object class which may be a relative address for a local class, a uuid for a global class, or any value that is unique/invariant for the entity instance in the stream such that a CREATE_UPDATE, DELETE and TOMBSTONE events all have the same entity-key. - For delete notifications the provider appends a record for the delete of the entity followed by a Tombstone record for the entity-key of the delete - Once stream population is sufficient, the provider changes the stream-state to “ACTIVE”. - The provider may be capable of making the stream “ACTIVE” prior to complete population of the stream. - In this state records will be streamed to any connected clients - New create/change/delete events will be appended to the log as defined in step (4) above. - The provider may set the stream-state to “PAUSED” at any point. - In this state no records will be streamed to connected clients. If the stream had changed to “PAUSED” from “ACTIVE”, a client may continue to receive records from the comms buffers for some time.

5.3.2 Use Case ST-1.2: Provider recovers a stream after internal loss

Number	ST-1.2
Name	Provider recovers a stream after potential loss of data
Process/Area	Streaming operation
Brief description	The provider suffers some loss of data from a source feeding a stream
Preconditions	The stream is in the available-stream list.
Type	Preparing and maintaining a stream
Description and workflow	1. The provider detects a potential loss of data from some area of the solution (the “area of concern”) that provides data to a stream. • This may be an internal component, perhaps where the provider solution is distributed, or an external feed. 2. The provider may set the stream-state to “PAUSED” or “ALIGNING”. 3. The provider will run an internal audit on the stream that ensures that the stream is populated with the correct state for the area of concern. If any cases of misalignment are detected the provider may set the stream-state to “ALIGNING” • For records of an entity that are found in the log feeding the stream where that entity is no longer at the source and most recent record is not of record-type “TOMBSTONE” the provider will append a Tombstone for the entity • For an entity that is found at the source but where there are no records in the log that is feeding the stream or there are only “DELETE” or “TOMBSTONE” records the Provider will append the details of the current state of the entity (taking advantage of approx.-date-and-time as necessary) • The provider will validate as appropriate that the latest record in the log for each entity that exist in the area of concern is aligned with the current state of that entity. If the entity in the log is not aligned the provider will append a record that reflects the current state (taking advantage of approx-date-and-time as necessary). 4. Once stream population is sufficient, the Controller changes the stream-state to “ACTIVE”. • The stream may have remained “ACTIVE” throughout the process. • In this state records will be streamed to any connected clients. • New create/change/delete events will be appended to the log. 5. The Controller may set the stream-state to “PAUSED” at any point. • In this state no records will be streamed to connected clients. If the stream had changed to “PAUSED” from “ACTIVE”, a client may continue to receive records from the comms buffers for some time. 6. As a result of the above process the client should be returned to a state of “eventually consistent” with the state of presented context with no need to take any specific action.

5.3.3 Use Case ST-1.3: Provider recovers a stream after an upgrade

Note that this use case is informative.

Number	ST-1.3
Name	Provider recovers a stream after an upgrade
Process/Area	Streaming operation
Brief description	The provider is upgraded
Preconditions	The stream is in the available-stream list.
Type	Preparing and maintaining a stream
Description and workflow	There are two distinct sub-cases: 1. The stream log persists through the upgrade: An approach similar to UC ST-1.2 could be taken where the stream log is updated with changes that occurred during the upgrade where an approach similar to UC ST-2.2 (“mark and sweep”) is used to clean up the stream. Again, approx-date-and-time could be used. 2. The stream log is lost during the upgrade: An approach similar to UC ST-1.1 could be taken. When the client connects with the previous token UC ST-0.3 will be run.

5.4 Client maintains alignment – Example strategies and approaches

This section provides further clarification of earlier use cases. The use cases in this section are informative.

5.4.1 Use Case ST-2.1: Client aligns with a stream

The client prepares for realignment during the initial alignment phase. This use case embellishes ST-0.3 and is provided as an example of a client solution.

Number	ST-2.1
Name	Client aligns with a stream
Process/Area	Streaming Infrastructure
Brief description	The client connects to provider and aligns
Preconditions	UC ST-0.1 has run successfully. The client has gained knowledge of the available streams via some mechanism (such as UC ST-0.2)
Type	Gaining and maintaining alignment
Description and workflow	- Client prepares to connect to a stream for the first time by - Setting current-stream-alignment-attempt-counter to 1 for the stream (stream-id) - Setting the alignment-attempt-start-time to current time. - Client connects to the stream and receives records (“simple update”) as per UC ST-03. - Client identifies the entity instance to add/update/delete using the entity-key (and potentially other data) and attempts to locate the entity using parent-address (see 3.10.1). - If record-type = CREATE_UPDATE - If entity exists, then update the entity and set stream-alignment-attempt-counter in the stored entity to current-stream-alignment-attempt-counter (stream id) - Update can be a simple overwrite, but client may want to mark changes or notify changes. - If entity instance does not exist, then create entity instance (with the alignment attempt counter value etc.) - If record-type = DELETE or TOMBSTONE (may be two records, both should be processed, may be able to make this efficient by expecting the TOMBSTONE if there is a DELETE) - If entity exists, then delete the entity. - If entity instance does not exist, then take no action. - Note: - The parent-address provides the positioning in the tree for the entity. The UUID uniquely identifies an entity and should be sufficient to locate the entity. In such a solution the parent-address is still necessary to provide the association to the grouping entities (such as Node grouping NEPs). - The entities identified in the parent address may not yet be present due to asynchronous arrival of stream information. Where this is the case, it is expected that a skeleton tree would be constructed to be subsequently filled in as the entities arrive. - Each time a record is successfully processed the client records the token in the token-from-latest-record-successfully-process-for-stream (stream-id) and the log-append-time-stamp for this alignment attempt - This assumes the client processes the stream records in the sequence received, if a more complex process is used, a more complex recording of tokens will be required. See also UC ST-4.2.

5.4.2 Use Case ST-2.2: Client realigns

Number	ST-2.2
Name	Client realigns
Process/Area	Streaming Infrastructure
Brief description	The client realigns after reconnecting to the stream.
Preconditions	UC ST-2.1 has run successfully
Type	Gaining and maintaining alignment

Description and workflow	1. Connection is dropped (triggered by client, by provider or by comms failure). 2. Client reconnects to the stream. This use case assumes that there is a resync triggered: • Provider driven resync (and very long comms failure) 1. Client provides the value (token) from token-from-latest-record-successfully-process-for-stream (stream-id)) in the connection request. • Client driven resync. 1. Client provides no token. 3. Provider streams from first record (offset 0). 4. Client detects realignment in first record received from the stream. • Client increments current-stream-alignment-attempt-counter for the stream (stream-id) and stores, for this alignment attempt, the alignment-attempt-start-time (set to current time), the token-from-latest-record-successfully-process-for-stream (stream-id) and the log-append-time-stamp from the first record just received 5. Client identifies the entity instance to update using the entity-key and parent-address (along with potentially other data) and attempts to locate the entity (“alignment update”) • If record-type = CREATE_UPDATE 1. If entity exists and has: 1. Newer log-append-time-stamp in the currently stored entity compared to the newly received record, then ignore record just received 2. Same log-append-time-stamp (and token) in the currently stored entity and the new record then store the newly received entity values and set stream-alignment-attempt-counter in the stored entity to current-stream-alignment-attempt-counter (stream-id) 3. Older log-append-time-stamp in the currently stored entity compared to the new record, then update (overwrite) the entity and set stream-alignment-attempt-counter in the stored entity to current-stream-alignment-attempt-counter (stream id) 2. If entity instance does not exist, then create entity instance as usual (with the alignment attempt counter value etc.) • If record-type = DELETE or TOMBSTONE (may be two records, both should be processed, may be able to make this efficient by expecting the TOMBSTONE if there is a DELETE) 1. If entity exists and has 1. Newer log-append-time-stamp in the currently stored entity compared to the newly received record, then ignore record just received 2. Same log-append-time-stamp in the currently stored entity and the new record, then delete the entity 3. Older log-append-time-stamp in the currently stored entity compared to the new record, then delete the entity 2. If entity instance does not exist, then take no action 6. Client receives further records. 7. Alignment has progressed sufficiently: • If log-append-time-stamp from the received record is (both of)
--------------------------	--

	1. More recent than the log-append-time-stamp-from-latest-record-successfully-process-for-stream (stream-id) for the previous alignment attempt (i.e., the client has passed the time of the last record successfully received before the realignment) 2. Less than the compaction delay from current time (compaction will not remove old records due to records that are newer than the compaction delay) • Then the client can sweep through the repository removing any entities still marked with previous alignment attempt counter value • Stream processing can continue uninterrupted as the sweep takes place (the client can revert to “simple update” processing as the sweep starts) 8. Once the sweep is complete the client has achieved “eventual consistency” with the provider
--	--

5.4.3 Use Case ST-2.3: Client performs a stream audit

Number	ST-2.3
Name	Client performs a stream audit
Process/Area	Streaming Infrastructure
Brief description	The client considers that there may be an alignment issue and decides to audit the stream.
Preconditions	UC ST-0.3 has run successfully
Type	Gaining and maintaining alignment
Description and workflow	Notes: 1. Whilst maintaining an existing connection to a stream, the client makes a second connection as described in UC ST-0.3 (1-3). 2. The client runs the second stream as if realigning and marks valid current state 3. When it passes the time of a current state, the entity is marked as suspect. Once sufficient progression has been achieved, the client will remove proved bogus records 4. When thing appears in the stream that is not in the repository, it is retained. Once sufficient progression has been achieved, the client will add the entity to the repository (avoiding race conditions with the main stream receiver).

5.5 Gaining and maintaining Alignment with individual network resources

A vast majority of the TAPI entities can be dealt with in the same way. The provider can offer streams that each include one or more classes. A stream will send records of all instances of all of the classes identified in its definition.

The use cases in this section are informative.

5.5.1 Use Case ST-3.1: Client maintains alignment with all instances of a class (e.g., Node) in a context

This use case considers a COMPACTED log solution.

Number	ST-3.1
Name	Client maintains alignment with all instances of a class (e.g., NODE) in a context
Process/Area	Streaming Operation
Brief description	The client discovers the availability of a stream for a class, connects to the stream aligns and maintains alignment ongoing.
Preconditions	Client knows which classes it needs to monitor and has run UC ST-0.1
Type	Gaining and maintaining alignment
Description and workflow	The client runs: 1. UC ST-0.2: From this the client has the address and connection method for an appropriate COMPACTED stream for the class(es) of interest (in this example NODE). 2. UC ST-0.3: From this the client achieves “eventual consistency” with the state of the class(es) of interest (e.g., NODE) including dealing with ongoing change. 3. UC ST-0.4: To ensure that the connection remains open. It is possible, during operation, that conditions in the network are such that any one of the following may occur. 4. UC ST-0.5: Where the client may lose some information fidelity but will maintain “eventual consistency” without any special action. 5. UC ST-0.6: Where the client and provider may need to deal with UC ST-0.9 6. UC ST-0.7: Where the client will simply suffer a short delay in receipt of information but will lose no fidelity or integrity. 7. UC ST-0.8: Where the client and provider may need to deal with UC ST-0.9 8. UC ST-0.9: Where the client will clean up its repository as it aligns with the provider

5.5.2 Use Case ST-3.2: Client maintains alignment with all alarms in the context

This use case deals with alarms that are processed, logged and streamed as described in sections 4.1, 4.2, 4.3 and 4.4. The alarm records abide by the model as described in 4.5 (further explained in 4.6). The alarm clear will be reported as described in 4.7. It is assumed that the event-time-stamp will be as described in 4.8 and that the optional normalization will perform as described in 4.9. The provider is also expected to deal with ensuring meaningful detection as described in 4.10.

Number	ST-3.2
Name	Client maintains alignment with all alarms in the context
Process/Area	Streaming Operation
Brief description	The client discovers the availability of a stream for Alarms, connects to the stream, aligns and maintains alignment ongoing.
Preconditions	Client knows which classes it needs to monitor and has run UC ST-0.1
Type	Gaining and maintaining alignment
Description and workflow	This is essentially UC ST-3.1 but where the specific class is <code>CONDITION_DETECTOR</code> 1. The client and provider cover UC ST-3.1.

5.6 Dealing with the whole context of resources

This section provides an overview of how the client would be expected to deal with all resources in a context. The use cases in this section are informative.

5.6.1 Use Case ST-4.1: Client maintains alignment with all resources in the context

Number	ST-4.1
Name	Client maintains alignment with all resources in the context
Process/Area	Solution Operation
Brief description	The client listens to a number of streams and assembles a view of the network
Preconditions	Client is running
Type	A Client building and maintaining the context
Description and workflow	1. The client runs UC ST-0.1 2. The client runs UC ST-2.x for all relevant classes and information • The client starts various streams in a sequence compatible with its alignment strategy 1. The streams guarantee order within any instance of a class but do not guarantee any ordering between instances of classes 2. For example, the client may decide to align nodes first • As entity information is received the client resolves references on-the-fly. 1. Where references do not resolve immediately, due to differential propagation delay etc., the client uses an appropriate method to defer the reference mapping until the relevant entity is received 2. As noted in UC ST-2.1 the tree referenced by the parent-address may not yet be fully present and hence a skeleton of the tree will need to be constructed and subsequently populated. • The client builds a view of interrelated network resources

5.6.2 Use Case ST-4.2: In a resilient solution the Controller a client is connected to becomes unavailable

Number	ST-4.2
Name	In a resilient solution the Controller a client is connected to becomes unavailable
Process/Area	Solution Operation
Brief description	The client detects the Controller to which it was connected is no longer available and connects to an alternative Controller instance for the streams of interest
Preconditions	Client is running UC ST-4.1
Type	A Client building and maintaining the context
Description and workflow	1. The client runs UC ST-0.1 for the new Controller instance 2. The client runs UC ST-2.1 for the new Controller instance

5.7 Connectivity Service Lifecycle

The use cases in this section are informative.

5.7.1 Use Case ST-5.1: Provide streams network changes to the client

Number	ST-5.1
Name	Provider streams network changes to the client
Process/Area	Solution Operation
Brief description	Changes in the network are detected by the provider and interpreted into changes to entities in the TAPI context.
Preconditions	Client is running UC ST-4.1
Type	Provider is monitoring the network for change
Description and workflow	1. The provider detected a change in network state and makes the appropriate adjustments to the TAPI context to reflect the change (e.g., creation of connections etc.) 2. The provider streams the changed entities via the appropriate streams to the client. 3. The attached client absorbs the stream maintaining eventual consistency with the network state. Note that the maintenance of eventual consistency is dependent upon UC ST-0.n.

5.7.2 Use Case ST-5.2: Client runs a provisioning use case ([LF TR-547] UC1x)

Number	ST-5.2
Name	Client runs a provisioning use case ([LF TR-547] UC1x)
Process/Area	Solution Operation
Brief description	Client provides connectivity-service details, and the provider takes action to satisfy the request.
Preconditions	Client is running UC ST-4.1
Type	A Client adjusting connectivity-services
Description and workflow	1. The client provides the connectivity-service details including routing constraints etc. 2. The provider assesses the request etc. as per [LF TR-547] UC1x. 3. The provider runs UC ST-5.1 and streams changes as they occur.

5.7.3 Use Case ST-5.3: Client runs the service deletion use case ([LF TR-547] UC1x)

Number	ST-5.3
Name	Client runs the service deletion use case ([LF TR-547] UC1x)
Process/Area	Solution Operation
Brief description	Client requests a delete of a connectivity-service
Preconditions	Client is running UC ST-4.1
Type	A Client adjusting connectivity-services
Description and workflow	1. The client requests the delete of a connectivity-service. 2. The provider assesses the request etc. as per [LF TR-547] UC10. 3. The provider runs UC ST-5.1

5.8 Message Sequence example

The hybrid message sequence diagram, in Figure 11 below, captures all relevant flows for the listed use cases. The diagram includes several special symbols that are highlighted in the key below the diagram. The diagram essentially lays out a sketch of a solution that would support the normative aspects of streaming. This section is informative.

It is assumed that the client has already used Restconf to get supported-streams and to get available-streams so as to have the connection method. In this example the connection method is assumed to be WebSockets.

In the diagram the behavior of the Device Source and TAPI Context are summarized. The diagram only shows presence and fundamental flow for these elements.

Two pipelines, the TAPI Context Pipeline and the Extended Pipeline, are shown in the diagram as yellow background rectangles. The TAPI context pipeline intentionally shows no detail as that is outside the scope of the interface definition.

The diagram shows coupled asynchronous parallel/concurrent repeating activities and independent asynchronous repeating activities each in a dashed box. Where the asynchronous activities are coupled there is a dashed line arrow showing the relationship as a ratio of activity, $n:1$, which indicates that there are potentially more trigger activities than result activities, or as a 1, which indicates that "eventually" there will be the same number of activities in both asynchronous elements (as a result of a flow through both). Some activities are shown as nested. Where nested there is an indication where there are n repeats of the inner asynchronous activity for each of the outer activities. Buffers/logs are shown to emphasize the asynchronous coupling. The compacted log is shown with a buffer/log symbol annotated with an "X" indicating compaction (the deletion of records in the log) and "0" indicating that the record is for all system time, i.e., for the time the system has been logging, (where compaction will remove duplicates and hence contain the log size).

The logs marked with "r" are limited to "r" records (size or number) and will block when that size is reached applying back pressure to the feed (via "fill"). Hence, in combination, there is a pipeline with backpressure from the client store to the provider log. This includes all sequences and flows in the "Extended Pipeline" shown as a large yellow rectangle.

To the left of the figure (i.e., to the left of the "underlying external comms" (brown vertical bar)) is the client side (which initiates the connections etc.), shown as a stylized example. The client is shown with both a database option and a compacted log option for storage. The critical features are the "Pong" and "Last commit". The majority of the client depiction is to explain last commit. Last commit is used by the client on reconnection to continue where it left off ("token~x").

The external comms between client and provider is shown as a brown bar and is not considered in any detail. It is assumed that it is reliable (e.g., TCP) and is playing an active part in maintenance of the pipeline.

The client token is opaque so the client has no knowledge of sequence through the token, although there is also exposure of the sequence number, this is primarily intended for stream analysis (note that it may be beneficial as part of normal behavior to validate communication).

The middle of the diagram shows the provider and explains the basic flows related to initial connection, loss of connection and forced connection drop.

To the extreme right of the figure are vertical progression bars that highlight phases of interaction between the client and provider.

The example functions (at the head of each timeline) can best be explained in terms of the phases of interaction in which they participate.

The “Prepare” phase involves an Authentication service, here shown within the provider system, which supplies Authentication Token on request to a Web Sockets (WS) client connection control function in the client. The Authentication Token (“Auth Token”) provided is used in a connection request in the “Connect phase”. Along with a null stream token (absence of a stream token). These exchanges are described in section 5.10 Message approach (WebSocket example) on page 67. This causes the stream to start from the oldest record (offset 0).

In the “Streaming” phase the provider continues to take records from the Compacted Log to the left and feed them into the communications system via appropriate queues that blocks on fill⁴³. In the example, the “Source and Process” function collects records from the log and ‘publishes’ them to “WS Endpoint Stream Server Source Actor” that fills a blocking queue feeding the “WS Endpoint Stream Server”. The “Websockets Endpoint Stream Server” feeding the underlying comms system (which is assumed to be reliable and blocking when full). On the client side, the stream client takes from the comms system and places on a queue. This results in there being an effective demand back to the Stream Server (stylized in the figure).

As noted above, the yellow background shows the whole (Extended) Pipeline. The intention is that this pipeline guarantees delivery to the client repository (shown as a compacted log “AND/OR” DB, but any repository relevant to the client is appropriate). The repository to the left of the diagram and associated functions are a simple sketch of the sort of operations that may take place. The key consideration on the left side is the “Last commit” token which records the last successfully stored in the persistent repository. The client benefits from recording this token as without it a full resync on comms fail would be required. The client need not record every token (as suggested by the “n:1” relationship).

The remaining phases, “Loss”, “Drop Connection” (two variants) and “Kill” pipeline are all related to failure modes. Assuming the client still exists and requires the stream information, in all failure cases, the client will reconnect using the stream token.

When a token is provided during the “Connect” phase the provider assesses the token to determine which record to start the stream from. If the token relates to a record:

- Newer than the compaction delay, then next record that was appended to the log is streamed.
- Between the compaction delay and the tombstone retention, then next record in the log, which is not necessary the next record that was appended due to compaction, is streamed
- Older than the compaction delay, then the provider forces a resync by streaming from the oldest record in the stream (Offset 0)

“Log and stream Control” monitors the state of each stream in the provider and drops the client connection when any problem is detected (e.g., it is blocked on a record older than tombstone retention).

Pong frames are required to maintain the stream when there is no activity, and a frame is required every 30 seconds (default time).

⁴³ For the pipeline to maintain integrity it is vital that back pressure prevents the queue from overflowing. The “blocking” is essentially through “demand”. If there is a queue overflow event, the pipeline must be restarted.

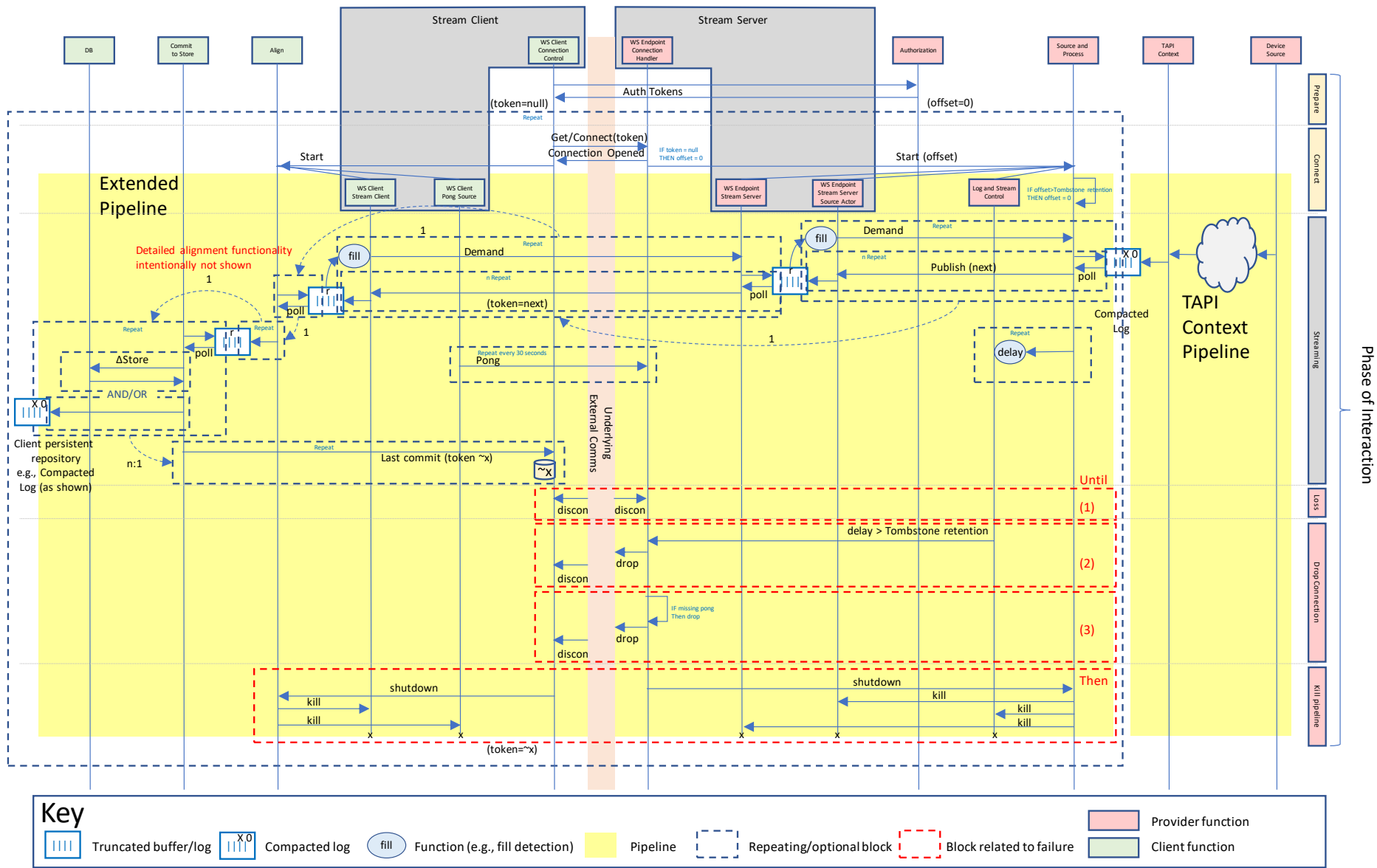


Figure 11 Hybrid Message Sequence Diagram for example implementation corresponding to Use Cases

The provider shall support:

- Authentication as described.
- A pipeline using backpressure from the communications system to ensure reliable delivery from the internal compacted log (full or emulated).
- Start streaming from:
 - The oldest record.
 - The record after the one with the stream token specified by the client in a connect request.
- Forced restart of stream from the oldest record when it detects:
 - A very slow client taking records older than Tombstone retention.
 - A reconnect request with a token for a record older than Tombstone retention.
- Pong frame timeout connection drop behaviour

The following figure shows the use cases and relationship to the hybrid message sequence chart in the figure above. The heading bars are the same as the righthand vertical bars on the previous figure. The flow across the figure assumes a chaining of the use cases in the order they are described in the earlier sections.

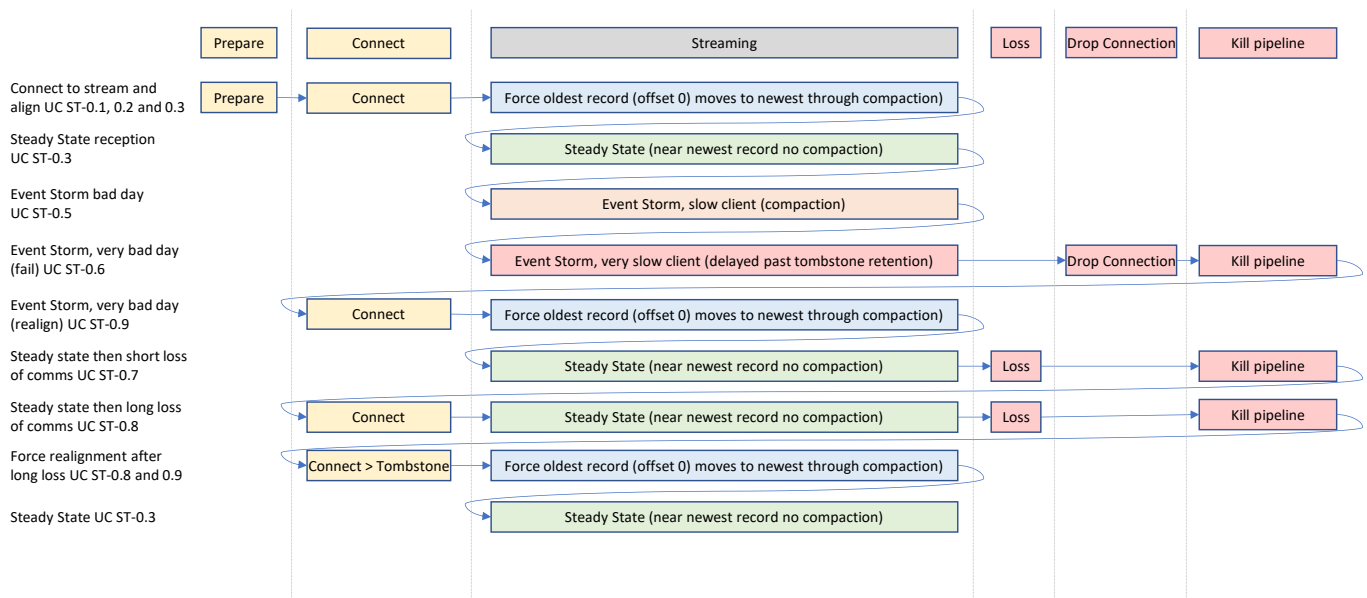


Figure 12 Phases of interaction for Use Cases

5.9 Use cases beyond current release

Beyond the current release there will be further improvements including the ability to dynamically adjust the stream behavior and the feeds to the stream as follows:

- Adjustment of compaction delay and tombstone retention on-the-fly
 - Allows for tuning of stream behavior at initial start-up and during predictable comms failures.
- Building and adjusting the context (creation, expansion, contraction and deletion)
 - Allows for multiple clients with differing needs and security clearances etc.
 - Negotiate Context opportunities based upon policy and client role etc.
 - Build explicit context (including topology, nodes, links etc.)
 - Various interactions to set up intent for nodes and links that themselves need to be realized in the underlying structure.
 - Note that the initial requests are in terms of shared knowledge such as city or building location and generalized termination points/flows with minimal technology detail.
 - In the general solution, with TAPI feeding a client, there could be several alternative contexts that can be provided. Contexts may focus on a single layer or layer grouping or on a region of the network etc. In a more sophisticated solution where there are many clients each with a slice, in these solutions various negotiations would be required to agree and form the context.
 - Note: Currently, the context is defined by a default intent where there was no opportunity for the client to express the context intent over an interface.
- Taking advantage of context adjustment capabilities to increase and decrease the intensity of view of information. This is applied where the intensity could not be handled across the whole context and hence is a focus. This may be where the parameter changes very often.
 - Spotlighting: Allows the client to selectively increase the fidelity of measurements by changing the measurement policy for a specific property and/or by including an instance of a property in the context where that property is usually not monitored
 - Single snapshot: Allows the client to select a property to take a momentary view of via the stream. This may be the capturing of a single counter value where that counter changes very often (e.g., a packet counter) such that streaming of the raw value would be excessive even for a single measure.

There are clearly other potential applications of a streaming solution where there are:

1. Many direct clients using the same context.
 - Here some form of multi-cast stream with a message broker or other multi-cast mechanism may be appropriate.
 - In the cases where there is a broker, it may be appropriate to continue to use the compacted log, but this will only benefit the broker and the provider will not be aware of clients or client performance challenges.
 - There are no apparent specific applications in a telecommunications network context.
2. Many direct clients each with different context
 - This leads to a characteristic very similar to that considered in this document.
 - The distinction is the multiple contexts.
 - It is likely that context build/modify will be necessary to enable this capability.
 - An application example is presentation of a slice to its client where each client has its own slice.
3. Short-lived clients that are attached for only a short period and want a specific context.
 - It is likely, for this case, that there will also be multiple clients.
 - It is possible that the short-lived clients define a short-term context and align with this.

- A possible case is a controller component dedicated to a particular test activity where the provider presents a context related to the test and when the test is complete the provider deletes the context.
 - Another possible case is a GUI. This does have a short-term cache and may benefit from ongoing updates even if only for a short period.
4. Clients that do not have a cache or store of information acquired from the provider.
- A solution that does not store information cannot be expected to take significant advantage of a stream that provides information on changes.
 - However, as the stream can be used to gain current state and the context can be set to define the relevant things that current state is required for, then a one-shot compacted log stream could be used to get a temporary snapshot.
 - A human driven CLI is an example, and this does not normally provide asynchronous delivery. In general, TAPI is not oriented towards this case.
 - A human tends to want to make somewhat random, complex and unbounded queries.

Cases 1-3 appear to benefit from the streaming capabilities described in this document, whereas case 4 does not, but it also does not seem to be a relevant TAPI application.

For each consideration in this section, it will be necessary to enhance the expression of capability such that the client can know what opportunities for adjustment are available. It is expected that this will be expressed using machine interpretable specifications.

5.10 Message approach (WebSocket example)

5.10.1 Basic interaction

The messaging is as defined below:

- The general structure for the WebSocket URL is obtained from the connection-address field of the discovered available streams. The implementations MAY support WS and WSS. For example, the URI can be of the form :

“wss://<host>/tapi/data/context/stream-context/available-stream=<uuid>”

where the uuid is acquired through a RESTCONF GET operation of available-stream (s). Using this URL would start the stream from the oldest record.

- Considering the "Connect (token)" from the message sequence diagram, this becomes “wss://<host>/tapi/data/context/stream-context/available-stream=<uuid>?start_from=<token>”. Omitting the token causes the provider to start from offset zero (i.e., the oldest record).
- In some cases, it may be relevant to start from the latest (e.g., for non-compacted logs) “wss://<host>/tapi/data/context/stream-context/available-stream=<uuid>?start_from=latest”.

5.10.2 Authorization example – WebSockets

The WebSockets specification indicates that the WebSockets server can use any client authentication mechanism available to a generic HTTP server (<https://tools.ietf.org/html/rfc6455#page-53>).

Use of the authorization framework defined in <https://tools.ietf.org/html/rfc6750> is recommended.

A valid authentication token for the provider must be supplied by the client via the use of "Authorization: Bearer <token>". This must be supplied in the header of the WebSockets connection request.

The authentication token is obtained from the provider using an appropriate method. This would supply a token for the specific username (i.e., the client).

5.10.3 Connecting to a stream example - WebSockets

This token is then used in the WebSockets handshake:

```
GET /tapi/data/context/stream-context/available-stream=<uuid> HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Host: <host name>
Origin: http://<host name>
Sec-WebSocket-Key: <key>
Sec-WebSocket-Version: 13
Authorization: Bearer <authentication token>
```

The provider will provide a response similar to:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Sec-WebSocket-Accept: eiUnNdCyox5gJ7eAbD4ZNo2H4xY=
Date: Mon, 07 Sep 2020 11:57:08 GMT
Connection: upgrade
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: default-src 'self' data: mediastream: blob: filesystem:
'unsafe-inline' 'unsafe-eval'
```

Followed by stream messages.

6 Appendix

6.1 Appendix – Considering compacted logs

The following section considers Kafka as an example implementation of a compacted log and then discusses implications of compaction and some storage strategies.

6.1.1 Essential characteristics of a compacted log

Compaction is described in the Kafka documentation at <https://kafka.apache.org/documentation/#compaction>

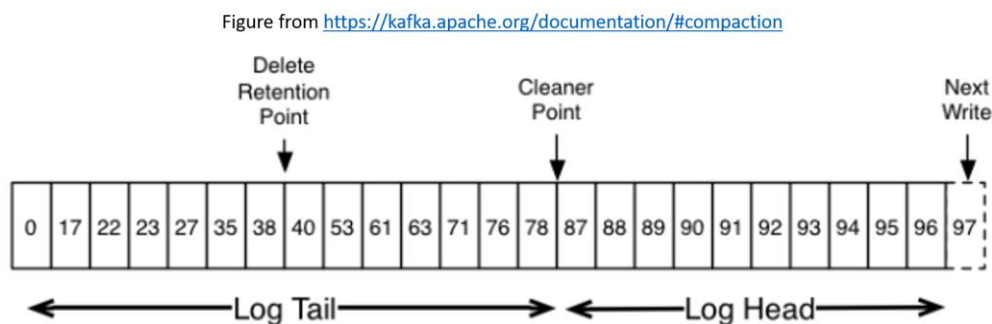


Figure 13 Kafka compaction

With retention (for non-deletes) set to “FOREVER”, the log becomes a single source of truth for absolute state (eventual consistency) and change of state (cost effective fidelity).

Client essentially reading next record in sequence:

- To the right of the Cleaner Point ensures full fidelity
- Between cleaner and delete retention (tombstone retention) points provides reduced fidelity but still supports eventual consistency
- To the left of (before) the delete retention (tombstone retention) point potentially violates eventual consistency and requires the client to go back to read record offset zero

6.1.2 Order of events

- Disordering of records
 - In a distributed system, information from the various parts is received with varying delay such that it is likely to be out of order
 - It can be assumed that time of day is well synchronized across the network
 - Event order can be regenerated (within reason) based upon time of event at source
 - Critical ordering that should be preserved through the log and pipeline is that related to each single event source. For example, consider an alarm detector.
 - It is possible that the time granularity at the source is not sufficient to resolve the active-clear sequence when cycling is very rapid as they can both appear to be at the same recorded time.
 - If the detector goes active and then clear, that ordering should be preserved through the system such that time granularity problems are not encountered, so that the view of system state is always eventually consistent with the state of the controlled system

- Multiple receipts of the same record: Idempotency
 - A record received more than once should not have any impact on system behavior

6.1.3 Log segments

A log may be divided into segments (units of storage) where the segments may have a defined maximum size. Records will be stored in an active segment until it becomes full at which point a new segment will be created for new records (a new active segment) and the previously active segment will become inactive. A log of this form will comprise a sequence of inactive segments and one active segment.

6.1.4 Partitions

To improve scaling a stream may be fed from a conceptual log that is formed from a number of separate logs where those separate logs are considered as partitions of the single conceptual log feeding the stream. The partitions have to be such as to maintain integrity of the flow with respect to the order of occurrence for any particular entity instance. Each partition has its own sequence numbering. The resource allocation to each partition and the load on the partitions is likely to be different. There is no particular order in which partitions may feed the stream.

6.1.5 Compaction in a real implementation

Considering compaction delay, in general system load will cause the compaction to sometimes drift such that less compaction occurs than is ideal.

In Kafka, compaction does not operate at a fixed cleaner point as the head (active) segment is not compacted. When the head rolls to become a tail (inactive) segment compaction can happen but may be delayed. The behavior is not fully deterministic as it depends upon segment fill and intermittency occurrence.

6.1.6 The Tombstone

The Tombstone record is a light-weight record that indicates that the identified entity is no longer in existence. The Tombstone is used to ensure eventual consistency during a realignment after a long communications failure. When the client receives a Tombstone, it should remove any record of the entity identified.

6.2 Appendix – UML Model

The Yang model for streaming has been generated using the Eagle tooling for the streaming UML model. The UML diagrams provide a convenient overview of the streaming structure and relevant properties. The key UML diagrams for streaming are provided in this section.

The conversion from UML to Yang accounts for various considerations including the change of formats and notations. For example, in UML camel case is used whereas in Yang uses kebab case (hyphenated lower case). The diagrams should be read with this in mind.

The figure below shows the structure of the streaming model. This structural view may help when reviewing the Yang representation.

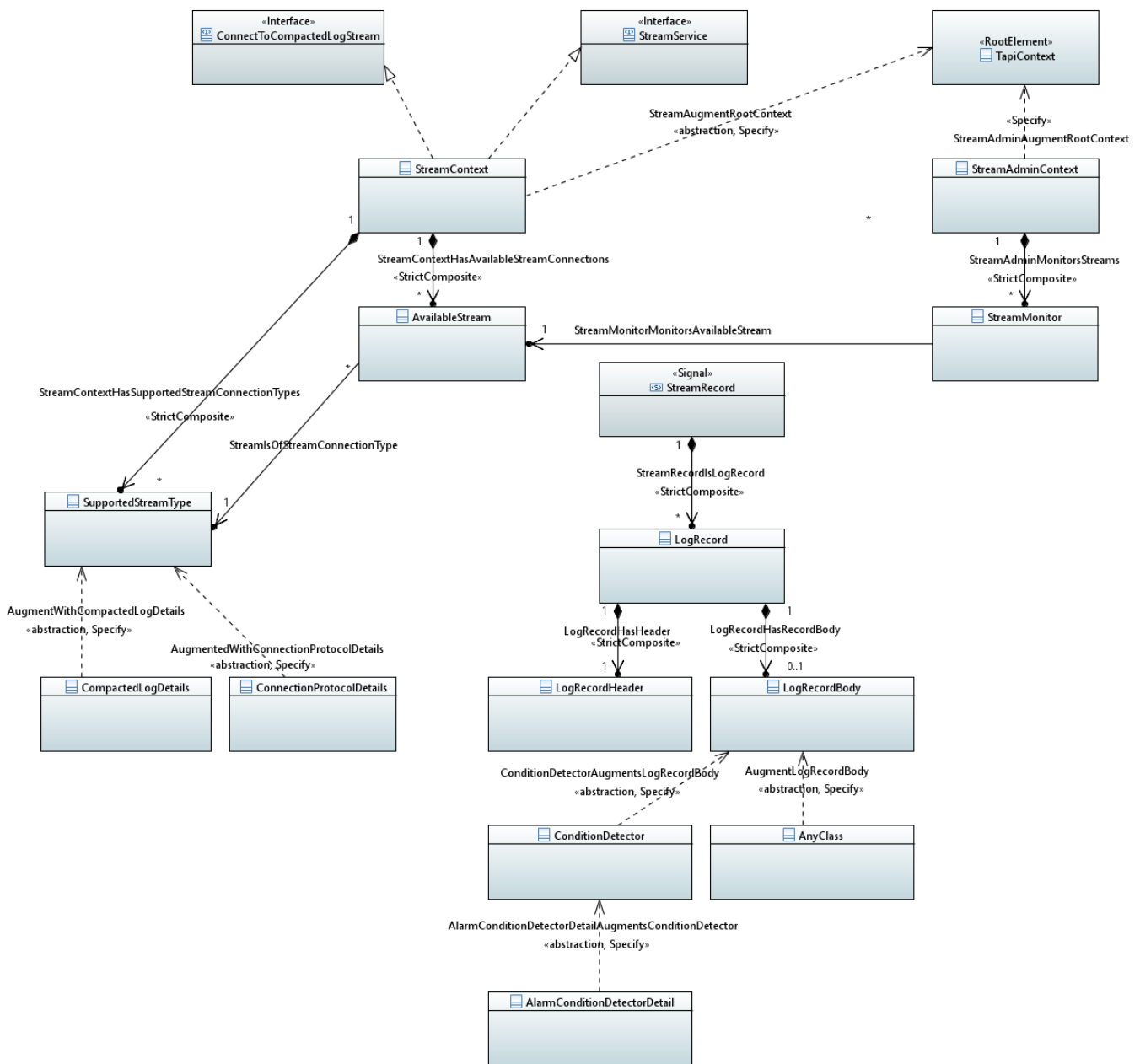


Figure 14 Structure of the streaming model

The figure below shows key content of the classes shown in the structure above.

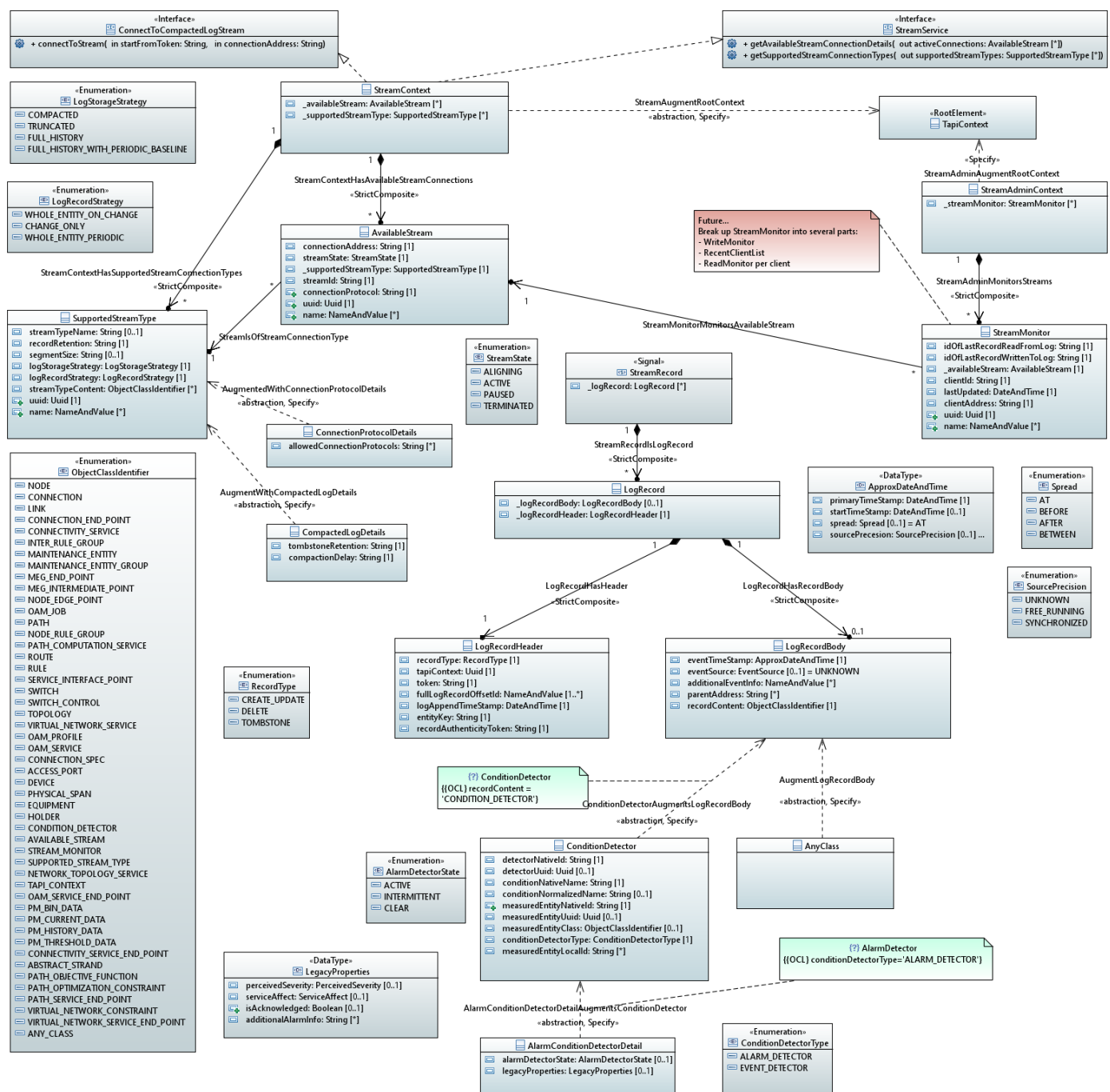


Figure 15 Structure and content of the streaming model

The figure below shows the UML form of augmentation (using the <<specify>> stereotype). All classes in the model can augment LogRecordBody.

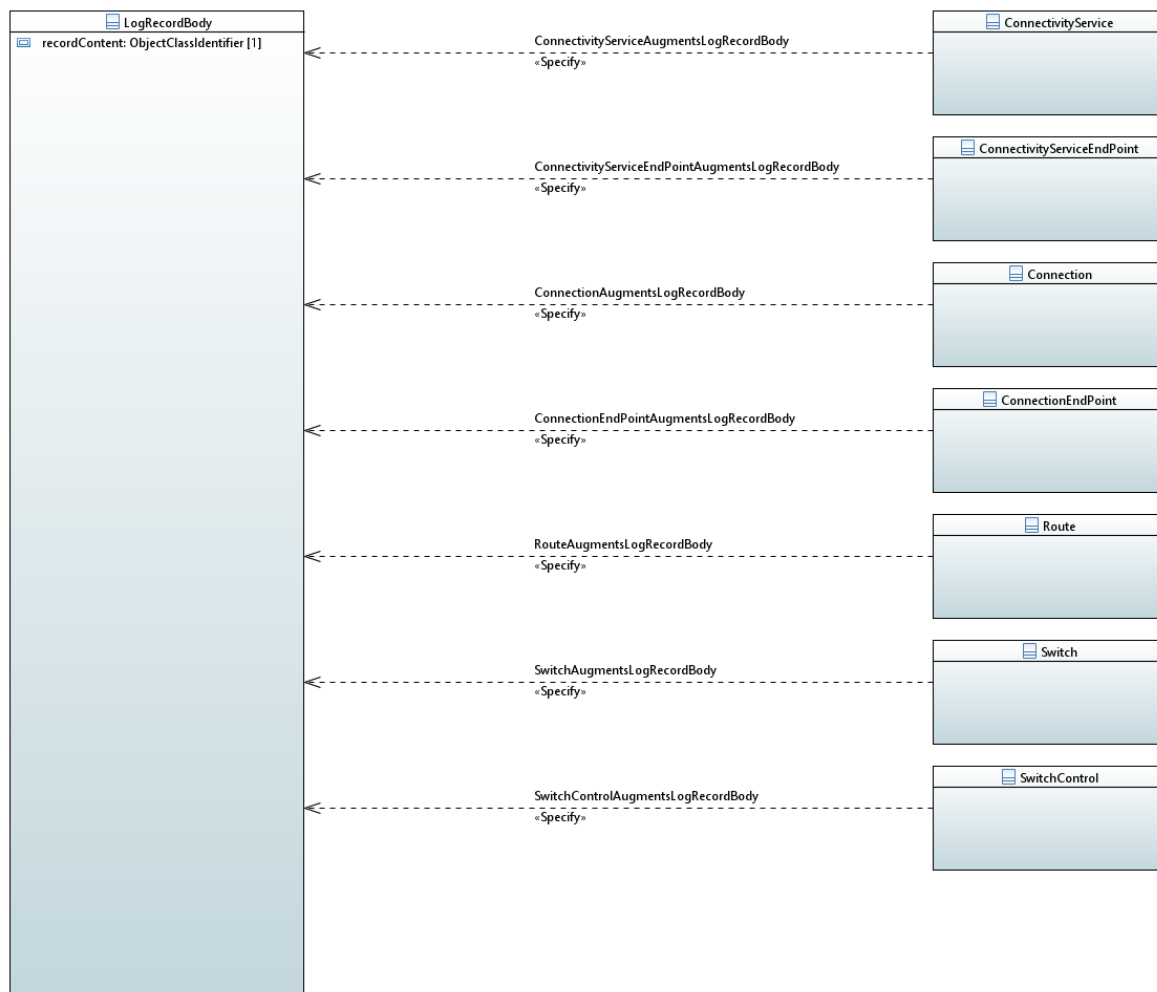


Figure 16 Example of Augmentation of the LogRecordBody with some classes from the model

6.3 Appendix – Stream record example

The following is an example of an alarm:

```
{
  "log-record-header": {
    "token": "62234255_0:-1379579382:AAAAQAahw4=",
    "log-append-time-stamp": "2021-04-28T14:14:28.780Z",
    "entity-key": "3669276387587255710",
    "record-type": "RECORD_TYPE_CREATE_UPDATE",
    "full-log-record-offset-id": [{
      "value-name": "partition",
      "value": "0"
    },
    {
      "value-name": "offset",
      "value": "910"
    }
  ],
  "log-record-body": {
    "event-time-stamp": {
      "approx-date-and-time": {
        "primary-time-stamp": "2021-04-28T14:14:20.000Z",
        "spread": "SPREAD_AT",
        "source-precision": "SOURCE_PRECISION_SYNCHRONIZED"
      }
    },
    "record-content": "CONDITION_DETECTOR",
    "condition-detector": {
      "detector-uuid": "b650df3b-7081-3447-91b3-166a59101122",
      "detector-native-id": "3669276387587255710",
      "alarm-condition-detector-detail": {
        "alarm-detector-state": "ACTIVE",
        "legacy-properties": {
          "perceived-severity": "CRITICAL",
          "is-acknowledged": false,
          "service-affect": "SERVICE_AFFECTING",
          "additional-alarm-info": ["CTP Loss of Frame, LOF"]
        }
      },
      "condition-native-name": "ODU LOF",
      "measured-entity-uuid": "6317c7ed-9f38-30bd-8f3b-4f6c64f74837",
      "measured-entity-native-id": "SiteA54:ODUCTP-1-A-15-2-4P1-2EP78",
      "condition-detector-type": "ALARM_DETECTOR"
    }
  }
}
```

6.4 Appendix – Detectors, detected conditions and alarms

This section deals with the conceptual model underlying alarm reporting.

6.4.1 Detect

As a result of observation of some properties of a thing (entity, assembly, environment etc.) there is a recognition of some relevant states distinct from the previously recognized states.

6.4.2 Detector

A device that both observes (monitors) a thing in a particular way and recognizes the state and change of state.

6.4.3 Condition

A specific combination of states that is defined and is of interest.

6.4.4 Condition detector

A device that detects a specific condition and reports the presence that condition (the detected condition). It also reports when that condition is no longer present.

The condition detector represents any monitoring component that assesses properties of something and determines from those properties what conditions are associated with the thing.

For example, a thing might be "too hot" or might be "unreliable".

The monitor may a multi-state output.

The condition detector lifecycle depends upon the lifecycle of the thing it is monitoring. For example, once the thing is removed/deleted, so is the condition detector.

6.4.5 Detected condition

The detected condition may be a state with associated values and with importance/priority etc.

6.4.6 Alarm

An alarm is an alert that relates to a specific detected condition that is considered problematic, where it is likely that some action will need to be taken to adjust system state such that the condition is cleared. Whilst the condition (alarm condition) is present, the alarm is **ACTIVE** and when not present the alarm is **CLEAR**.

6.4.7 Alarm detector

A detector that detects and highlights a specific alarm condition by reporting that the alarm condition is **ACTIVE** and the absence of that condition by reporting that the alarm is **CLEAR**.

6.4.8 Traditional alarm reporting and legacy-properties

The legacy-properties are provided to deal with the traditional alarm reporting properties. Alarm systems of the 20th century were based primarily on local lamps (initially filament bulbs) and bells. Lamps can only be on or off, and bells sounding or not sounding, so alarms were Boolean in nature. Where a detector was essentially multi-state it was converted into multiple Boolean statements.

The management of the network devices (equipments) was essentially human only and local only (there were rarely remote systems). The network device with the problem was the only possible indicator of importance and it had only three distinct bulbs to illuminate (filament bulbs tend to fail requiring costly replacement).

The network devices were relatively simple in function and analysis of the detectors was crude. There was only the network device to indicate severity. The network device also could provide the best view as to whether a service was impacted, although clearly it had almost no knowledge.

In a modern solution with well-connected remote systems that increasingly analyse problems and where there is increasingly 'lights out' building operation, the network device's guess at severity etc. is irrelevant. In addition, with sophisticated resilience mechanisms, the network device cannot make any relevant statement on whether the customer service has been impacted.

Likewise, in a world where there were no remote systems and local management was the only practice, alarms had to be locally 'acknowledged'. Where there are remote systems, per alarm acknowledge is burdensome.

7 References

[DDD]	https://en.wikipedia.org/wiki/Domain-driven_design
[GNMI]	https://github.com/openconfig/reference/tree/master/rpc/gnmi
[GNMI-SPEC]	https://github.com/openconfig/reference/blob/master/rpc/gnmi/gnmi-specification.md
[KAFKA]	https://kafka.apache.org/
[ONF TR-512]	https://www.opennetworking.org/wp-content/uploads/2018/12/TR-512_v1.4_OnfCoreIm-info.zip (also published as [ITU-T G.7711])
[LF TR-547]	https://github.com/Open-Network-Models-and-Interfaces-ONMI/TAPI-Documentation/tree/v2.1.5/ReferenceImplementationAgreements/TR-547 v 1.2
[RESTCONF]	https://tools.ietf.org/html/rfc8040
[RFC6455]	https://tools.ietf.org/html/rfc6455 The websocket protocol
[WC3 SSE]	https://www.w3.org/TR/eventsource/ Server-Sent Events
[YANG]	https://tools.ietf.org/html/rfc6020

8 Definitions and Terminology

No terms are special to this work.

This document uses terms defined elsewhere. In some cases the term is used in a particular way in this document. Particular usage is highlighted.

CEP	Connection End Point (a TAPI class) [LF TR-547]
Client	In this document, an application/system (usually a controller (see below)) which uses TAPI streaming (hence is a TAPI client) to gain and maintain alignment with the current state of a network that is exposed through a provider system The client is a superior controller to the provider
Control	To attempt to achieve an agreed intent and measure the results to validate achievement and where necessary to initiate actions to fix any problems Note that automated management is control (see [ONF TR-512] (specifically TR-512.8))
Controlled network	A controlled system that consists of devices that provide networking functionality.
Controlled system	A system of devices that are managed/controlled by a controller. Also covers managed system as automation of management is the focus and automated management is control
Controller	A devices that controls other devices. - Examples, with varying degrees of control responsibility and varying degrees of automation, include devices mentioned in this definition list such as SDTN Controller, SDN-C, Orchestrator, EMS, NMS, OSS (see Figure 1 Example SDN architecture for WDM/OTN network on page 7) - The primary focus for Streaming in TAPI 2.1.3 is simple and efficient ongoing alignment of a Controller (client) with a view presented by another Controller (provider).
DDD	Domain Driven Design

Device	A thing formed from an assembly of electronic (and mechanical) equipment usually running software that is made or adapted for a particular purpose. See also: • network device • controller
EMS	Element Management System
ForwardingDomain	An ONF Core Model class [ONF TR-512]
GUI	Graphical (human) User Interface
KAFKA	https://kafka.apache.org/
LF	Linux Foundation
Log	A sequential store
Management	(see control)
Management-Control	(see control)
MEP	Maintenance End Point (a TAPI class) [LF TR-547]
Network device	A device that is designed to and/or is used to support networking functions (e.g., an OTN Switch).
NMS	Network Management System
Node	A TAPI Class [LF TR-547]
OAM	Operations Administration and Maintenance
ONF	Open Networking Foundation
Orchestrator	An overarching Controller that coordinates other subordinate controllers
OSS	Operations Support System
PoC	Proof of Concept
Provider	In this document, an application/system (usually a controller (see above)) which offers TAPI streaming (hence is a TAPI provider) to enable a client to gain and maintain alignment with the current state of the network that it, the provider, exposed • The client is a superior controller to the provider
SDN-C	Software Defined Network Controller [LF TR-547]
SDTN	Software-Defined Transport Network
SDTN Controller	An SDTN Orchestrator [LF TR-547]
TAPI	Transport API (Application Programmers Interface) [LF TR-547]
Tombstone	A special message that indicates that an entity no longer exists.
Topology	A TAPI Class [LF TR-547]
TR	Technical Report (from LF)
UML	Universal Modeling Language
UUID	Universally Unique Identifier
WS	WebSockets
Yang	Yet Another Next Generation... see [YANG]

9 Individuals engaged

9.1 Editors

Nigel Davis	Ciena
-------------	-------

9.2 Contributors

Nigel Davis	Ciena
Ramon Casellas	CTTC
Arturo Mayoral	Facebook (Arturo was previously at Telefónica)
Kam Lam	FiberHome
Pedro Amaral	Infinera
Jonathan Sadler	Infinera
Karthik Sethuraman	NEC
Andrea Mazzini	Nokia
Ronald Zabaleta	Telefónica
Malcolm Betts	ZTE
Xiaobing Niu	ZTE
Jai Qian	ZTE

10 Appendix: Changes between versions

10.1 Changes between v1.1 and v1.2

- Updated to Linux Foundation document.

End of Document